ReSearchOS: A Reference Architecture for Sustained Human–AI Research

Technical report · Not peer reviewed

Shiyu Hu*
School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore
shiyu.hu@ntu.edu.sg

30 October 2026

As execution becomes easier to delegate, researchers must decide what is worth asking, what is sufficient grounds for judgment, and what is worth retaining.

Abstract

In human–AI research that runs for weeks or months, later work builds on earlier judgments. Yet unless they are recorded, the reasons for those judgments and the conditions under which they apply tend to be lost when a conversation ends, and the sources they rest on can change. Suppose a paper the researcher relied on is corrected weeks later. The existing judgments can neither all stand nor all be discarded: (1) what the correction leaves intact should be kept with its grounds; (2) only the part it touches needs updating; and (3) what the evidence cannot yet settle stays unresolved until the researcher answers. Telling these apart takes an understanding of the evidence and cannot be recomputed automatically as in a build system, so the decision belongs to the researcher, while the system finds the judgments recorded as resting on that source and hands them back. On this basis the report presents ReSearchOS, a research environment owned by the researcher and kept mainly in local files. It keeps judgments with their grounds and scope, and an AI model's output remains a candidate until the researcher admits it. The report gives the design principles, definitions that make the three operations checkable and a reference architecture, together with a command-line prototype and a downloadable version to try. At its next command the prototype notices a changed source, registers a recheck of the related judgments and refuses a candidate whose source changed during review; by default the downloadable version only collects candidates and does not change the researcher's records. The prototype implements part of the design, and its effect awaits a study of use and a comparison with simpler approaches.

Keywords: human–AI collaboration; research-state maintenance; reference architecture; judgment continuity; extensibility.

*This technical report describes the design principles, the definitions of research state and the command-line prototype of ReSearchOS, a personal project that the author designed and built independently. It has not been peer reviewed, and the prototype is partial; its formal parts are there to make the design precise enough to discuss, not to establish reviewed results. It is shared for understanding and discussion.

1 Research state, not research files

In human–AI research that runs for weeks or months, a researcher works across many sessions, sometimes with several people, and later work builds on earlier judgments. The sources behind those judgments change over time, so getting the current step right is not enough: what is carried forward also has to be checked again.

The information that check needs is easily lost. Unless they are recorded, the reasons for a judgment tend to disappear when a conversation ends. The files survive, but they do not say how a conclusion was reached, under which conditions it applies or what remains unresolved. Suppose a researcher reads a paper, concludes that method A is better than method B and writes down "use A". Weeks later the paper is corrected, and no record shows which conclusions rest on it. The same week the researcher's student asks an AI assistant in another session to read the corrected paper, and the assistant concludes that A will also be better in a situation the paper never tested. Whoever continues from that one line has nothing else to go on.

Fuller records help, but they cannot settle what should follow a change, because a judgment is not a function of its sources. In a build system an output is a function of its inputs: when an input changes, what depends on it is rebuilt and no one is asked (Figure 1a). One correction, by contrast, can leave part of a judgment standing, make another part need revision and leave a third undecidable, and telling them apart takes someone who understands the evidence (Figure 1b).

A system can therefore find the judgments recorded as resting on a changed source and hand them back, while the decision stays with the researcher. For the same reason, what an AI model or another session produces remains a candidate until the researcher admits it (Figure 1c); neither they nor the files should make these choices for the researcher. What is admitted is a judgment, not a file, and it has to carry its grounds and limits, so what has to be kept is research state: those judgments and what is still unresolved, in files the researcher owns.

What the researcher has to decide can be seen in the paper itself. The paper ran two benchmark tests and found A better on both, but it tested only one kind of situation and says nothing about whether A stays ahead in another, a question left for an experiment the researcher plans to run later. The correction finds the numbers in the second test wrong, and with the right numbers A's advantage on that test disappears.

After the correction, the parts of the conclusion have to be treated separately. The first test was not wrong, so its support for the conclusion still stands and is **kept**. The second test changed, so the part of the conclusion that rested on it is **updated**, and nothing else is touched. The untested situation still has no answer, so it is put on **hold**: it stays unresolved, left to the planned experiment. The assistant's claim, which nothing in the paper supports, is refused. Keep, update and hold all ask the same question: what does the new evidence touch?

From here on the conclusion is called J. The two methods are visual tracking methods, which follow a target through a video. The paper tested them only while the target stays in view, so in full J reads "A is better than B while the target stays in view", and the untested situation is the one in which the target leaves the view and comes back. The report calls these two situations the in-view condition and the re-appearance condition, and Figure 1 draws the example in the notation used throughout the report.

This gives a fuller account of research state: what has to survive such a change is the question, the evidence and its relations, the accepted judgments with their scope, the open conditions and the supported next step. A note saying only "use A" loses all of this, down to the open condition and the experiment that comes next. This is the report's answer: what has to be kept is research state, not research files, and when a source

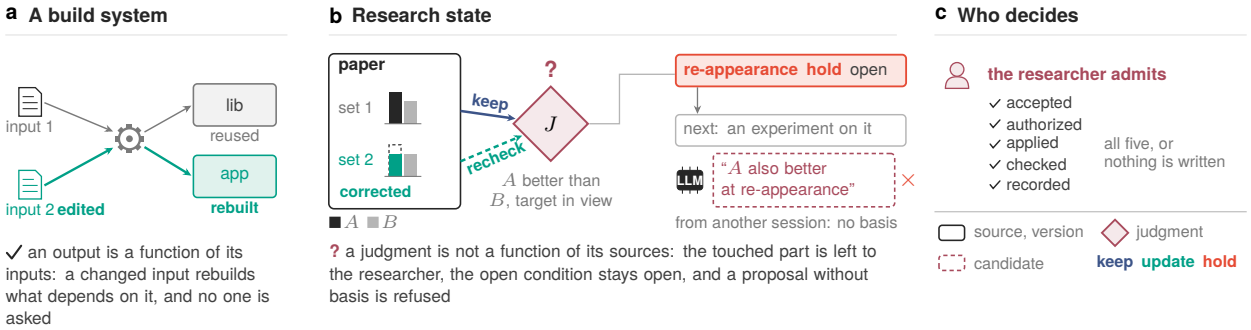


Figure 1. A build system rebuilds on its own; research state waits for the researcher. **a**, A changed input rebuilds dependents. **b**, Correcting the second test (set 2) keeps one ground of J, rechecks the other and leaves the condition unresolved; bars are shapes, not data. **c**, Only admission writes. Squares: sources; diamonds: judgments; dashed: candidates; blue, green, red-orange: keep, update, hold.

changes, the researcher, not the AI model or another session, decides which part is kept, updated or left unresolved.[1]

Research state changes in three ways, and the example has all three: time passes as the paper is corrected and sessions end, and tools and AI models may be replaced; conditions change when the purpose turns to re-appearance; and participants change when another session proposes and someone must decide what counts as correct. Faced with such changes, the tools researchers already use can keep and update records, and some flag a changed source, or a claim that rests on it, to a person. However, none of the classes compared in Section 6.1 keeps a judgment with its scope and basis version and hands only the part a change touches back to the researcher to decide; that section runs the same correction through each of them.

ReSearchOS is a researcher-owned, local-primary environment built for the moment a source changes. Its single rule of authority is the one reached above: machine outputs begin as candidates, that is, outputs not yet admitted, and only the researcher admits a candidate into research state. Admission, the step that turns a candidate into research state, needs that decision and keeps a receipt, the record of what was done and checked (Section 4.1).

The name **ReSearchOS** comes from Re Search: returning to material with a new question and retaining what that examination changes.[2] In the environment, researchers and computational assistants form questions, examine evidence and revise judgments across sessions through shared research objects. We call the aim it serves judgment continuity: recovering the grounds of earlier work and reconsidering whether they still apply. The environment keeps the shared records and the unresolved work, while the work itself happens in agent sessions, at the command line or in specialist tools, which the environment does not orchestrate.

For the researcher, the rule takes the form of one loop: recover the research state that matters now, advance the research in whatever client does the work, and confirm only what deserves to enter research state. Recovery selects the minimum context for the work in hand rather than the whole repository, and confirmation asks what real change the session produced that the future should inherit, writing nothing when there is none. Section 2 shows the loop as the six commands of one session.

[1] The author studies similar questions in other work, on tracking systems and on learners. This report applies the question only to research practice and does not present the environment as a result of that other work.

[2] Other public projects use the name ResearchOS, including `github.com/CognitiveAISystems/ResearcherOS` and `github.com/JRH89/researchos`; ReSearchOS is not related to them.

1.1 Three contributions

The first contribution, in Section 3, is a set of design principles stating what the environment must preserve so that the choice among keep, update and hold stays with the researcher: human judgment, conditional composition, records that outlast tools and correctable accumulation.

The second, in Section 4, is a set of definitions that make the principles checkable. They describe research state and what keep, update and hold each require of maintenance work, one subsection apiece (4.1–4.3). On this basis they compose the three within bounded goals, units of maintenance work with a frozen scope and an explicit outcome (4.4).

The third, in Sections 2 and 5, is a reference architecture and prototype that carry the definitions into the commands a researcher actually runs. At its next command the prototype notices a changed source and registers a recheck of the judgments that rest on it. At the terminal it admits a narrowed judgment only on the researcher's decision, as a new judgment block beside the original; it refuses a candidate whose source changed during review and will not overwrite another's edit. A reader can download a version to try, which is copy-only by default: it collects candidates but never admits them.

> **How to read this report.** Section 2 shows one session, Section 3 says why the environment is built as it is, Section 4 gives the definitions, Section 5 sets out how the prototype is built and its gaps, and Section 6 compares neighboring systems and makes the case against. A first pass covers this section, Section 2 and the principles in Table 3. A reader deciding whether to try the environment should start with Section 6.2, which makes the case for replacing it with a strong AI model that reads and writes files directly and is given a one-page versioned handoff; if that handoff wins, ReSearchOS should shrink to its smallest form. Appendix A lists the objects of the definitions and the lifecycle of a requirement, and Appendix B is the implementation record. Supplementary Note 5 states every definition precisely and argues the two propositions, and Note 6 lists every gap.

Table 1 gives the plain meaning of the words a first-pass reader meets before Section 4. Section 4 explains held and waiting on the example where it first uses them; Table 8 in Appendix A lists the terms with their states, and Supplementary Note 5 defines them precisely. Most names match the prototype's code and commands.

Table 1. Terms used throughout the report

Term	Meaning in this report
Research state	What research carries forward: the question, the evidence and its relations, the accepted judgments with their scope, the open conditions and the next step (Section 4.1)
Client	Any entry point where the work is done, such as the command line, the Dashboard or an agent session; every client works against the same records
Card	A Markdown record, such as the one that carries *J*
Candidate	An AI model's output or a draft that has not been admitted
Admission	The step by which a candidate enters research state; it needs the researcher's decision (Section 4.1)
Receipt	The record of what was done and checked
Ground	A reason a judgment rests on, such as the first benchmark set for *J*
Basis	The recorded field of a judgment that pins a source at a version, written `Basis:`

Term	Meaning in this report
Requirement	A registered piece of maintenance work (Section 4.2)
Recheck	The requirement that asks whether a judgment still stands after a source it rests on has changed, as in the recheck of J (Section 4.2)
Check	A test on a requirement, with one of three results: pass, fail or cannot yet tell, written 1, 0 and ? (Section 4.3)
Bounded goal	A unit of maintenance work with a frozen scope, a frozen specification and an explicit outcome; in the prototype a session approximates it, and the code calls it an episode (Section 4.4)
Downloadable version	What a reader can install, copy-only by default: it collects candidates but never admits them. The author's prototype adds admission and Dashboard writeback

Claim boundary. Section 4 is a specification: its two propositions apply only under the hypotheses stated with them in Supplementary Note 5. Sections 2 and 5 describe a partial prototype at the commits listed in Appendix B.3, and the session in Section 2 shows what those commits do. That prototype enforces the researcher's decision at the terminal only (gaps 1 and 2; gap numbers refer to Section 5.4 and Supplementary Note 6); Section 5.4 lists what else it does not yet do.

As for the evidence, the prototype's one user so far is the author, who also built it and evaluated every result reported here, so none of those results is independent of the author. No study of use has been run, nor a comparison with a strong AI model given a one-page handoff, so the report does not establish human acceptance or research benefit. The environment is meant for researchers who already check sources and work across sessions; how novices would learn with it, or how a team would settle disagreements in it, needs separate study.

2 A session after the paper is corrected

The session follows the example of Section 1; the methods A and B are fictional, and no real method's performance is reported. Figure 2 draws it in six steps, each one command of `ros.py`, the program that runs a session. Beforehand the researcher had bound J, "A is better than B while the target stays in view", to the paper at version v_1, grounded in both benchmark sets; the unknown under the re-appearance condition was among the open conditions, and the re-appearance experiment was planned. Then the paper is corrected to v_2: the second set's numbers change and the first set's do not.

The correction reaches the records when the user, or an agent the user instructs, replaces the local file that records the paper, here `sources/paper.md`, and the next `ros` command sees its changed content digest. The session opens with `ros today`, which sees the version change and registers the recheck of J as open work. It asks no question yet. The correction may only fix a typo and leave the judgment valid, so whether to keep or update is still undecided.

Next, `ros context` delivers the permitted sources and the current research state with a receipt, and `ros run judgment --target <card> --executor copy_packet` writes a packet for the card carrying J without calling any AI service. The packet is a Markdown file carrying the task and the delivered context, which the user pastes into any assistant. `ros collect <run> --file draft.md` then reads the assistant's draft back as a candidate. Only the one to three judgment blocks in the draft are read, and anything else is ignored:

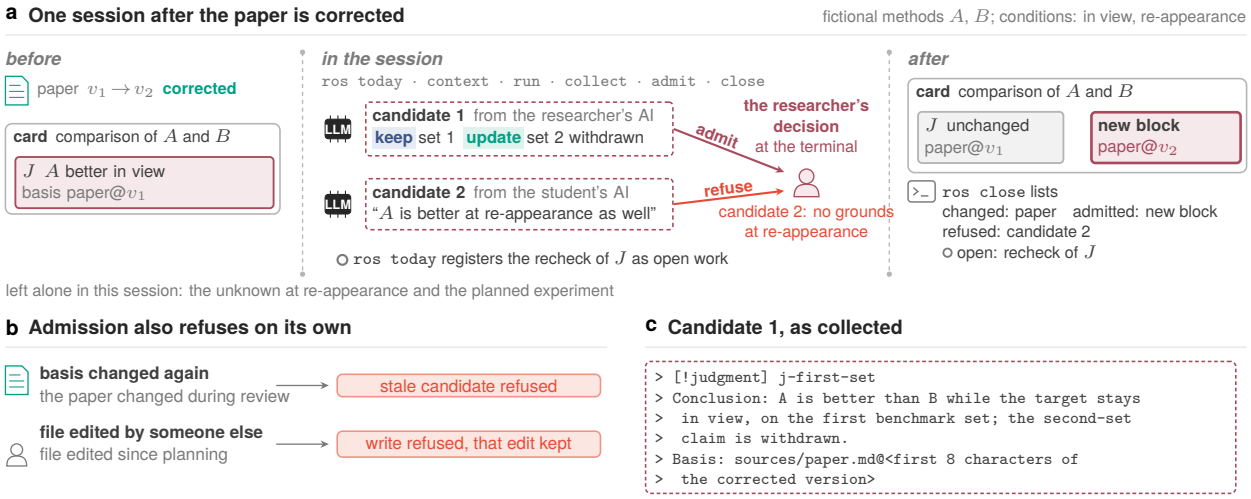


Figure 2. In one session the narrowed judgment is admitted beside *J* on the researcher's decision, the unsupported proposal is refused, and the recheck stays open. The methods *A*, *B* are fictional. **a**, The session, from before `ros today` to after `ros close`. **b**, The two cases in which admission refuses on its own. **c**, The first candidate collected.

```
> [!judgment] j-first-set
> Conclusion: A is better than B while the target stays in view, on the first benchmark set; the
 ↪  second-set claim is withdrawn.
> Basis: sources/paper.md@<first 8 characters of the corrected version>
```

Here the basis pins the paper at v_2. Because a basis is recorded per source file, as its path with a content digest, the prototype cannot see the split between the first and second sets inside the paper; the candidate states which part the correction touches and the researcher decides (gap 11). A basis that names no version is pinned to the version current at collection, and a draft pinned to an outdated version is refused. The candidate leaves the re-appearance condition and the experiment unchanged, because they have independent grounds: withdrawing part of *A*'s advantage implies neither that *B* is better nor that the experiment should be cancelled.

The researcher then decides, and `ros admit <candidate>` performs admission. Only after the researcher's review does it write the narrowed judgment, as a new judgment block beside *J*, and because the decision was confirmed at the terminal it is recorded as the researcher's. This step is what update means in the prototype: *J* stays on its card, still pinned to v_1, keep and update are both written in the new block beside it, and the prototype does not edit *J* itself.

In the same session a second candidate, "*A* is better after re-appearance as well", is collected from the student's AI assistant, with its basis at v_2. The paper reports nothing about re-appearance, so the researcher refuses it at the terminal. The refusal is recorded as the researcher's decision; nothing is written and research state does not change. The session ends with `ros close`, which compares the sources at its start and end and lists what changed, what was admitted, what was refused, which candidates are still undecided and what is still open.

Besides the researcher's refusal, admission can also refuse on its own, in two cases, and neither refusal is a scientific judgment. If the paper changes again during review, admission rejects the candidate as stale and asks for a new proposal on the current material. If someone else edits the file carrying *J* after the write is planned and before it lands, the content the write was planned against no longer matches, and the system

refuses to overwrite that edit. An edit made during the review itself is kept, because the write is planned only after the decision, against the file as it then stands.

All of this is the author's prototype. The downloadable version, in its default copy-only mode, runs the same commands on its own fictional fixture, passed with `--root`, so its runs are not evidence for this session. As Table 2 shows, it may write runtime records, packets and candidates locally, but it yields only copyable candidates or planned differences and never admits a candidate or changes the researcher's sources:

Table 2. The worked session and the downloadable version by default

Command	The author's prototype in this worked session	Downloadable version by default
`ros today`	Observe the source change and mark the judgment for recheck	Inspect fictional sources and maintain git-ignored working state
`ros context`	Deliver permitted sources and reconstructed research state with a receipt	Deliver permitted fictional context with a receipt
`ros run judgment --target <card> --executor copy_packet`	Prepare a task packet without calling an AI service	Prepare a copy packet; provider dispatch remains refused
`ros collect`	Return the draft as a candidate pinned to its source version	Collect a candidate and do not admit it
`ros admit`	Admit after the researcher's terminal decision; refuse a stale basis, or a file changed since the write was planned	Show the planned difference through copy fallback, without writing it into the source
`ros close`	List changes, admissions, refusals, undecided candidates and open work	Record the local session and its remaining work; admit nothing

Set against Section 1, the session's results fall under three heads. The researcher kept the first-set result, which carries on in the new block with its in-view scope, and updated the conclusion by withdrawing the second-set claim in that same block. Hold here amounted to making no change: the session leaves the re-appearance condition and the experiment unchanged, the prototype does not register them (gap 12), and the refused proposal did not change them either. At `ros close` the recheck of J is still listed as open.

3 What a research environment must preserve

Section 2 left the decision about J with the researcher, and a research environment has to be designed so that it stays there. The design starts from one sentence: research is decomposed until a judgment can be made, then composed back into research state that can be continued. When research state changes, what this sentence asks for is exactly keep, update and hold: what still stands, what must be revised and what must wait for an answer.

Each of the three operations can go wrong, and each of the four principles guards against one way this happens; they follow the life of a judgment as it is made, reused, carried across tools and corrected. Where efficiency, control and retention pull against each other, the principles put four things first: consequential human judgment, conditional composition, records that outlast tools and correctable accumulation. Figure 3 traces how they lead through the definitions and the architecture down to the code.

The principles set these priorities, not uniform directories, methods, interfaces or review arrangements. Because explanations take time to read and effort to maintain, and review can interrupt work or harden into habit, inspection is concentrated on the transitions that matter, and routine work goes on within the scope already agreed.

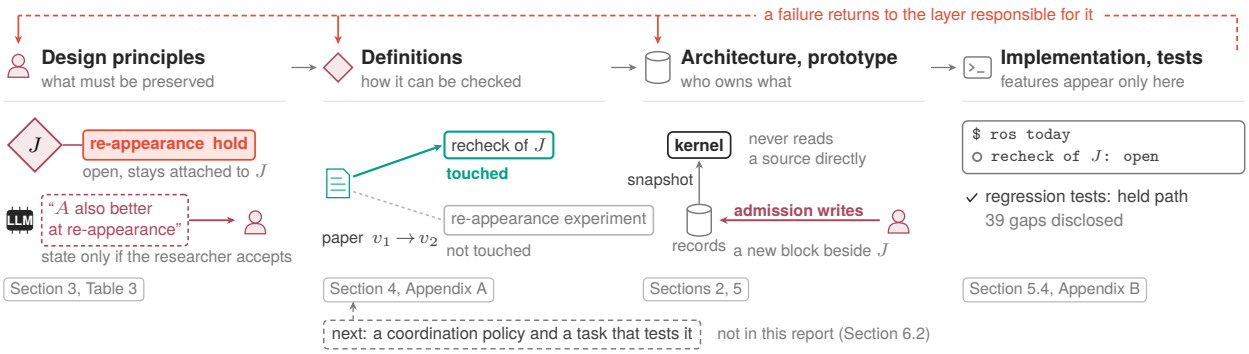


Figure 3. Principles, definitions, architecture and implementation each answer one question, and a failure returns to the layer responsible for it. Each layer shows what it says about the running example, and gray tags give its place in this report. The coordination policy and a task that tests it are the next step and are not in this report.

The principles come from the author's own design history, and their rules are written in six internal design documents; this section summarizes those documents and has been checked against them. Table 3 gives each principle's question, commitment and mechanism, Supplementary Note 1 gives the examples and the sources behind each principle, and Supplementary Note 3 restates the provisions in full with their owners.

Table 3. The four design principles and where they are carried

Principle	Question it answers	Commitment	Mechanism
Human judgment	Who accepts a judgment?	Only the researcher decides what becomes research state; machine outputs begin as candidates	Candidate, decision, change, check and receipt recorded separately
Conditional composition	When may a result be reused?	A successor starts only when the predecessor's actual result establishes its conditions; unknown does not count as satisfied	Rule applications with declared premises and checks, composition (Section 4.4) and three-valued checks
Records that outlast tools	What survives tool replacement?	Readable, versionable local sources remain authoritative while AI models, clients and plugins can be replaced	Source ownership, context and disclosure receipts, and a basis pinned to a source version
Correctable accumulation	What changes after correction?	Correction narrows affected claims without erasing history; failure, unknowns and unverified states remain visible	Judgment revision, rollback receipts and capability evidence accumulated across sessions

3.1 Only the researcher turns a candidate into state

The first failure is keeping what was never accepted. In the example, the student's AI assistant proposes that *A* is also better after re-appearance; admitted without the researcher's decision, that proposal would widen *J* to a case the paper gives no grounds for. Therefore only the researcher decides what becomes research state. Admission carries this principle: a candidate enters research state only when all five conditions of admission hold, the first being the researcher's decision (Section 4.1). Admission settles who accepted a candidate, however; whether the candidate is right is still a matter for later checks, since agreement does not establish scientific correctness.

3.2 A successor runs only on an established result

The second failure is updating on an unknown. In the example, success while the target stays in view may motivate the re-appearance experiment, but recommending *A* after the target leaves would update on exactly the unknown the experiment must settle. Therefore a successor starts only when the predecessor's actual result establishes its conditions. To that end, composition carries conditions, unknowns and failures forward (Section 4.4), and a three-valued check returns pass, fail or cannot yet tell, where cannot yet tell never counts as satisfied (Section 4.3). Fields that match between two steps let information pass between them, but they show neither that it is valid nor that its use is permitted.

3.3 Records outlast the tools that wrote them

The third failure is losing what should have been kept. If *J* and its grounds lived only in the conversation that produced them, nothing would show that the later correction of the paper touches *J*. A record should still make sense after the conversation window closes and the plugin that produced it stops running, so readable, versionable local sources remain authoritative, while AI models, clients and plugins can be replaced. Accordingly, a view that shows work as complete does not make it accepted, and a cache never becomes a second source of facts.

In the prototype, *J* sits on a card with its basis pinned to a source version; context is delivered with a receipt (Section 2), and adapters keep no second store of facts (Section 5.1). Retention is not retrieval: a record is kept with its conditions of use so that later work can judge whether it still applies.

3.4 Correction narrows a claim; it does not erase history

The fourth failure is updating by deletion. In the example the correction touches only the second benchmark set, yet deleting *J* would erase both its first-set ground, which the correction leaves intact, and the record that the second-set claim was once accepted. Correction therefore narrows the affected claims without erasing history: failures, unknowns and unverified states stay visible, and what changed, why it changed and which parts remain useful are all kept. The narrowed judgment is admitted beside *J* (Section 2), and the recheck of *J* leaves the unresolved set only through a recorded resolution or cancellation (Section 4.2). Moreover, a single success establishes no rule, and the report assumes no benefit to the researcher's understanding.

4 What keep, update and hold require

The principles say what must be preserved but not yet what counts as preserving it. This section gives definitions that make them checkable: it states keep, update and hold in words on the example of Section 1 and follows them through the correction of the paper. Supplementary Note 5 states every definition precisely and argues the two propositions of Section 4.4; Appendix A sets the objects side by side and draws the lifecycle of a requirement; and Appendix B.1 gives each definition's implementation status.

4.1 Keep: research state and admission

To resume the comparison weeks later, the researcher needs five things rather than the files themselves: the question being answered, the evidence used and how it fits together, the judgment accepted and its scope, the conditions still unresolved, and the step planned next. In the example these are the choice between *A* and *B*, the paper's two benchmark sets, the judgment *J* for targets in view, the unknown under the re-appearance condition, and the experiment on it.

Research state is these five, reconstructed from the records, and its unresolved conditions also record conflicts and the cases in which a judgment does not apply. Because research state is a reconstruction rather than a summary, its sources and status can be checked and corrected; how much it holds depends on what the question needs, not on how much material exists.

How research state changes when a source changes is what keep, update and hold answer. A judgment whose grounds the change does not touch is **kept**, with its grounds and scope. A judgment whose grounds are touched also stays, but a **requirement**, a registered piece of maintenance work, marks it for recheck (Section 4.2); it is **updated** only when the researcher admits a candidate that narrows or replaces it, pinned to the changed version of the source. In the example, then, J is updated, not deleted: its ground on the first set is untouched, and the admitted candidate narrows it.

Admitting that candidate does not by itself close the recheck of J, however, because a requirement also carries **checks**, tests it must pass, and the recheck of J must still pass its check (Section 4.3). Nor do these definitions compute which ground a change touches: keep and update are followed for each ground only through that recheck.

Hold has two places, and the definitions keep them apart. A scientific condition that the evidence cannot settle stays unresolved in research state, as the re-appearance condition does throughout the example. A step of maintenance that cannot yet be decided is held, with a question to the researcher; by these definitions the recheck of J is held after the correction (Section 4.3).

Three parties act on research state (Figure 4). The environment, ReSearchOS in the figure, selects context from the history it may use, reconstructs the research state and presents it with any candidate; an AI model proposes an output and a change; and the researcher decides what happens next. Whichever party starts it, every change has the same shape: select inputs for a purpose, check that the change may start, apply it, and check the result. Following typed graph transformation [31], this report calls such a change an **application** of a rule, and the rule declares what it reads, what it must preserve and what it intends to produce.

This shape is used because different errors are repaired at different steps (Figure 5). A misread number in the paper, for example, calls for correcting that observation and what depends on it, whereas a ranking for targets in view, given in answer to a question about re-appearance, calls for clarifying the purpose first. Only an omission that recurs across tasks is grounds to propose revising the rule itself, and later tasks must then check the revision.

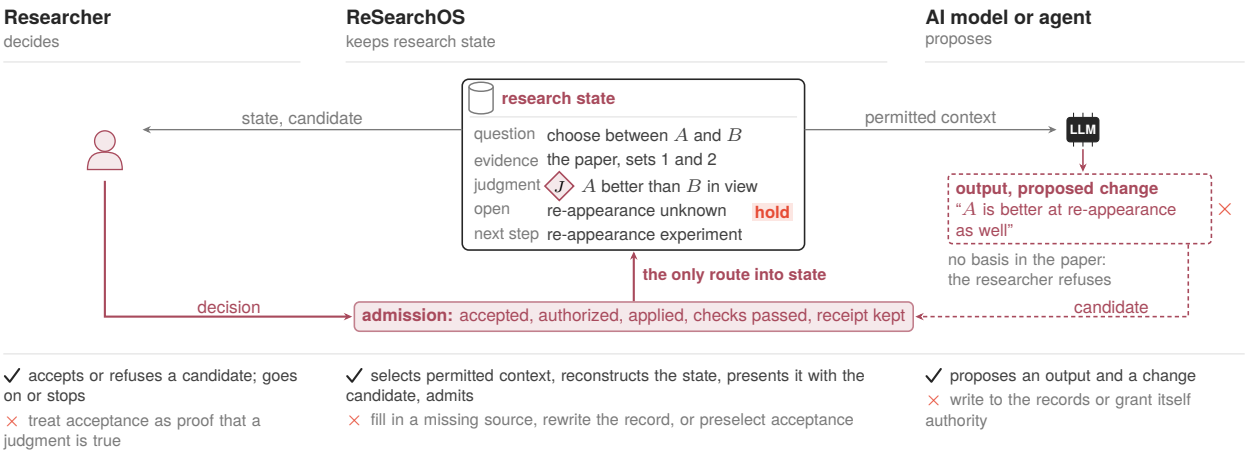


Figure 4. Only the researcher's admission turns a candidate into research state. An AI model or agent receives permitted context, and its output and proposed change return only as a candidate: here the proposal about re-appearance from the student's AI assistant, which the researcher refuses. The lines at the foot give what each party does and may not do.

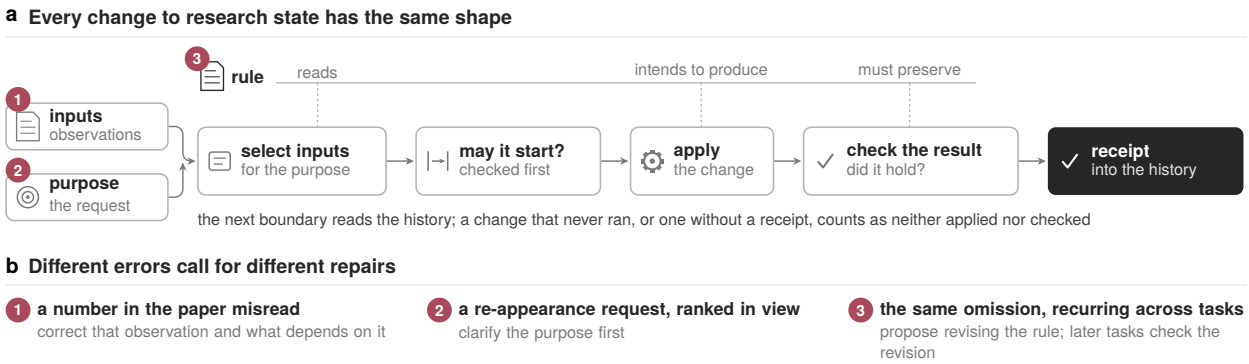


Figure 5. Every change to research state has the same shape, and different errors call for different repairs. a, The steps of a change, with its receipt kept in the history the next decision boundary reads. **b**, The three errors of the text, each numbered by the step of **a** where its repair applies.

A candidate enters research state only through **admission**, and no single party can take that route alone. Five conditions must hold together: the researcher accepts the change; a trusted host or an actual instruction within scope authorizes it, which the candidate's own text cannot do; the change has actually been applied; its checks have passed; and a **receipt**, the record of what was done and checked, is kept in the history that the next decision boundary reads (Section 4.2). A change that never ran, or one without a receipt, therefore counts as neither applied nor checked. The researcher's acceptance settles the operational decision, not whether the judgment is scientifically true.

4.2 Update: what a source change touches

Weeks after the comparison the paper is corrected from version v_1 to v_2. Two things then have to be settled: which version of the paper is now current, and which work the change touches. The system settles both at **decision boundaries**, the moments at which it looks at what has arrived and decides what to do next; work goes on between them. Each boundary has a cutoff, and a record that arrives after it waits for the next boundary. In the prototype a change is seen at the next `ros` command (Section 2).

The current version is not simply the newest file. An object's **current version candidates** are its versions with no known successor among the records the task may use, so two versions neither of which is known to follow the other stay side by side as unresolved branches, and the choice between them is left to the researcher (Figure 6a). The prototype identifies versions differently (Section 5.4). In the example the answer is plain: once the correction is seen, the paper's current version is v_2.

Which work the correction touches depends on what each requirement records, and the recheck of J shows how. Like every requirement, it has a stable identity and is registered for a purpose, which carries the scientific conditions of the work and what its review relies on. It also records the inputs it depends on: here the paper, together with the versions of the rule and purpose behind the recheck. A change **touches** a registered requirement when one of those inputs has a new current version since the previous boundary. The recheck of J depends on the paper, so the correction touches it; the re-appearance experiment rests on the unresolved re-appearance condition rather than on the paper, so it is not touched (Figure 6b).

Registered requirements are not the only way work is found: when work is first set up, or a scoped check is rerun, a scan also finds needs, missing dependencies included. Supplementary Note 5 gives the remaining rules, for comparing versions, late-arriving records and scans. Once touched, a requirement joins the **unresolved set** and leaves it only through recorded resolution or cancellation. Deferral, waiting and an

unsupported report of completion do not remove it, so work that nobody examined survives to the next boundary.

Touching therefore marks what to inspect, not what to edit. It follows recorded dependencies only and cannot discover a dependency that was never recorded. Had the correction fixed only a typo, the recheck of J would still be touched with nothing in J to change; conversely, a new study can make J unreliable without changing any input J depends on. A changed file is a reason to look, not a verdict.

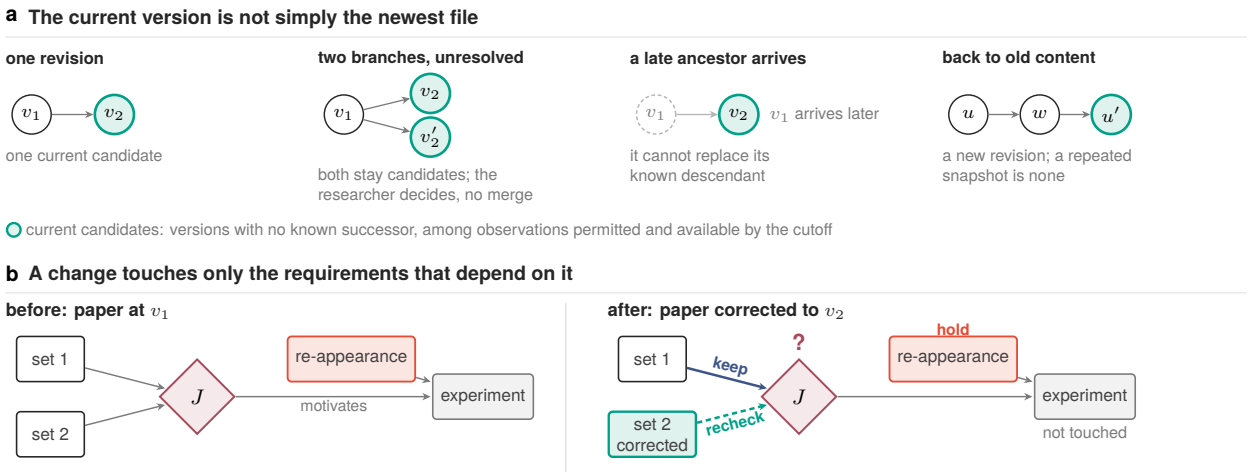


Figure 6. A change touches only requirements that depend on it or that a scan finds, and the current version is not simply the newest file. a, Four cases: one revision, unresolved branches, a late ancestor that cannot replace its descendant, and old content restored as a new revision. **b**, The example before and after the correction, in Figure 1's notation.

4.3 Hold: three-valued checks, held and waiting work

After the correction, whether the recheck of J can be closed depends on whether the result on the first set still supports J, and only the researcher can judge this. The system therefore gives each check one of three results: pass, fail or cannot yet tell, written 1, 0 and ?. The result is pass or fail only when a qualified check that is currently valid establishes it; when the evidence is missing, stale or in unresolved conflict the result is cannot yet tell, and a ? never counts as a pass. A requirement is **resolved** only when all its checks pass on the same current receipts. This is why the old receipt for v_1 cannot answer for v_2, and so cannot close the recheck of J.

Sometimes every check that has not passed returns cannot yet tell, and only the researcher can settle each of them. The system then asks the researcher one question for each, with a fixed set of options and only once for each version of the check's basis, and the work stays there until the answers come. This is what **held** means; a check that fails rules out holding. For the recheck of J, an answer that J is still supported closes that check, and an answer that it is no longer supported reopens the recheck; without an answer the recheck stays held and is never counted as done.

Asking once per basis version, and the prototype's `ros today` gathering the unresolved questions into one list, are meant to keep the load of review down and keep review from becoming a habit (Section 3). Neither has been tested, however, and even a correction that only fixes a typo opens a recheck.

Work can also be **waiting** rather than held: it waits on a trigger that will bring it back, because the researcher deferred or narrowed it or because something outside the goal's scope blocks it. Deferring the re-appearance experiment to its scheduled date, for example, would make it waiting. Figure 7 shows what each combination

of checks allows, Figure 18 in Appendix A draws the whole lifecycle of a requirement, and Supplementary Note 5 states the other limits on checks, holding and waiting.

Held and waiting work stays in view, but neither resolves nor cancels a requirement. Only resolution or cancellation removes it from the unresolved set, and cancellation is a separate recorded decision to stop pursuing the need, which does not count as a repair. Even that closure is not permanent: a later change to the paper can touch the same requirement again.

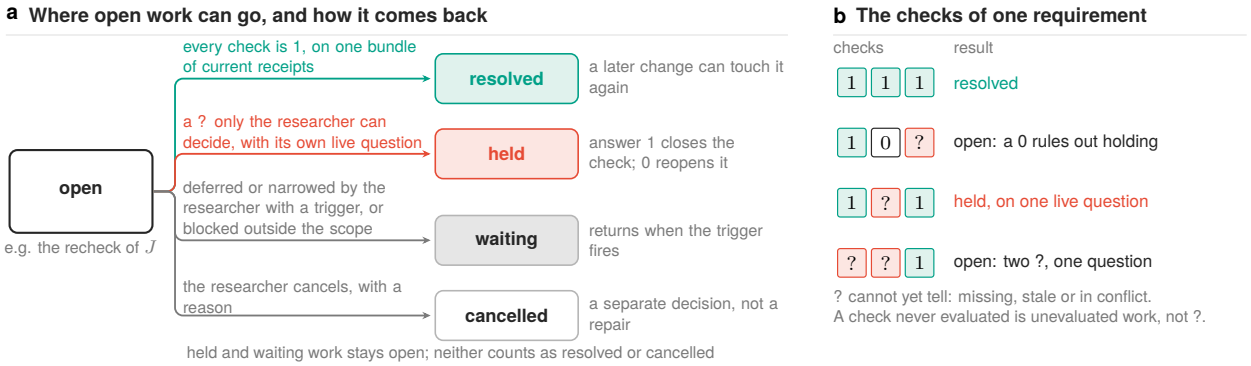


Figure 7. Only checks that all pass can close a requirement; otherwise it is held on a question, waits on a trigger, is cancelled by the researcher, or stays open. **a**, Where open work can go; the right column says how each state returns. **b**, Combinations of 1 (pass), 0 (fail) and ? (cannot yet tell) and their result.

4.4 Composition and bounded goals

With the recheck of J held and the experiment still ahead, two questions remain: when one step may build on the result of another, and when the work is done. The example answers the first: success while the target stays in view may motivate the re-appearance experiment, but it may not be used to recommend A once the target leaves the view. In general, one application may build on the result of another only when two sets of conditions are both established for the purpose: the conditions the first result carries, and those the second requires of its input.

A condition is established only when the same current receipts pass every check the purpose declares for it, which is the same test as resolution. In the example the result is established only for targets in view, so it can motivate the experiment but cannot support recommending A after re-appearance (Figure 8). An unchecked condition is not false, but it cannot authorize the next step.

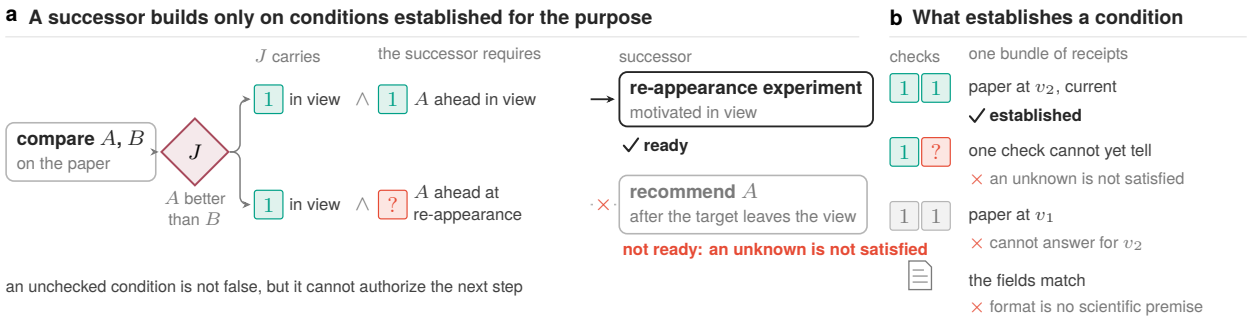


Figure 8. A successor builds only on conditions established for its purpose. **a**, What J allows: success in view may motivate the experiment; recommending A after the target leaves the view is not ready. **b**, What establishes a condition. An unknown is not satisfied, receipts for v_1 cannot answer for v_2, and compatibility of format is not a scientific premise.

The second question, when the work around the recheck of J is finished, is harder than it sounds: in the early prototype, issues kept coming back after a cleanup was declared finished, across ten cleanup rounds and nearly two hundred commits. The definitions therefore confine the work to a **bounded goal**, a unit of work with a frozen scope, a frozen specification and an explicit outcome (Figure 9). The specification freezes the question, the purpose, the scope, the versions of rules and checks, what must be preserved, the authority, the budget, and the source versions with their cutoff, and it is admitted like any other candidate. Completion is then judged against something that cannot move during the run.

The scope, within this specification, is what the goal may read. Ownership is a tree that gives each record one owner, following where the record lives, while dependency can link any records. The goal covers the unclosed records of its project's node and the nodes below it, plus a few inputs from other projects read through the interfaces those projects declare, which are frozen with the specification. A read that bypasses an interface lies outside the frozen scope (Supplementary Note 5).

Within a bounded goal, being done and stopping are different things. The goal is **Done** only when everything still unresolved in it is either held on a question or waiting on a trigger, and none of the three non-exempt kinds of work remains unresolved. These are in-scope work the run never evaluated, an earlier specification that cannot be reconstructed, and a confirmation still owed for a direct edit that bypassed admission (Supplementary Note 5). Those three may be held or waiting for a while, but they must close before the goal is Done, so a goal cannot end by accepting its own conclusion by default.

The goal stops when it is Done, when a part of its budget runs out, or when the researcher decides to stop, and what it reports depends on how it stopped. Done with nothing unresolved is complete. A goal can also be Done with held or waiting work still unresolved, as in Figure 9b, and then it is complete_with_held: it lists that work with its questions and triggers, and reports the share of the goal's registered requirements that are resolved; a zero share is reported as deferred, not as completion. Otherwise, a goal stopped by its budget ends budget_stopped, and one stopped by the researcher ends human_stopped; neither reports completion.

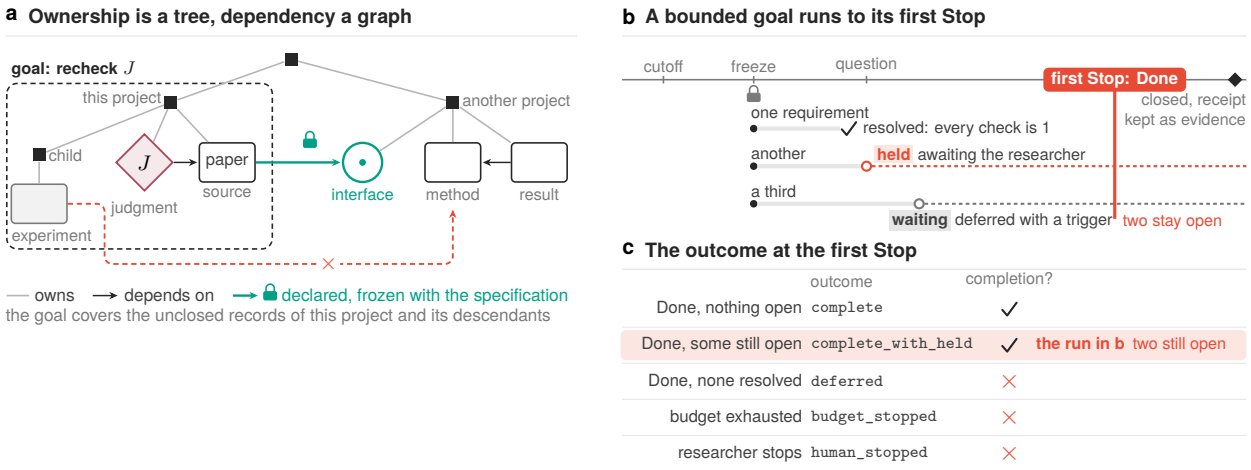


Figure 9. A bounded goal freezes what it may read and must answer, and reports held or waiting work as open. **a**, Scope: ownership (gray) and dependency (arrows). **b**, A general run to the first Stop: one requirement resolved, one held, one waiting; the last two stay open. **c**, The outcome, complete_with_held; closed with a receipt, the goal is kept as evidence.

Two propositions state what these definitions guarantee, and Supplementary Note 5 gives their hypotheses and arguments. The first says that goals whose frozen scopes do not overlap share no requirement, so neither's completion depends on the other: a goal on an unrelated project, for instance, neither adds to nor removes from the requirements of the goal that rechecks J. The second says that a goal either reaches Done or

exhausts its budget in finitely many steps; besides its hypotheses it needs a condition on scheduling, because reopening a requirement, or an answer of "unsatisfied", adds unresolved work.

Both hold only under these conditions, and a finite budget of decision boundaries guarantees that a goal stops, not that it completes. Nor does either proposition show that completion can be reached: that would need every unresolved requirement to be decidable within scope and budget, with no new source change.

5 The prototype: how it is built and what it does not yet do

The prototype that ran the session of Section 2 does not implement all of the definitions of Section 4. Part of keep, update and hold, and part of the researcher's decision among them, now run as commands. This section therefore describes how the prototype is built together with how far it stands from those definitions, and the description holds at the two commits listed in Appendix B.3, where their tests are summarized.

5.1 Responsibilities and the stable kernel

Section 3.3 requires that AI models, clients and plugins can be replaced while the researcher's sources stay authoritative. To meet this, Figure 10 maps a target division in five rows: the clients, the methods, the stable kernel, the adapters and the records. In this division the researcher owns the sources, and the stable kernel computes changes and unresolved requirements from snapshots that adapters provide and never reads a source directly. Adapters handle files, execution and receipts through existing services without creating a second store of facts, and in this division they write only staged edits with receipts. Clients organize presentation; an AI model's output can propose but cannot grant write authority, so it enters only as a candidate.

The prototype still differs from this target division. The session kernel that runs the commands of Section 2 is `ros.py`; behind it, the contracts of governance (what may become research state) and of coordination (what needs attention now) share one package. However, admission itself is still carried out by `ros.py` rather than by that package, so the figure's single stable kernel is a target, not existing code (gap 3). Among the clients, agent sessions include Codex, Claude Code and others, and the configuration is `config/ros.json`. Supplementary Note 2 holds the commands, the record tables and the stepwise records behind this section.

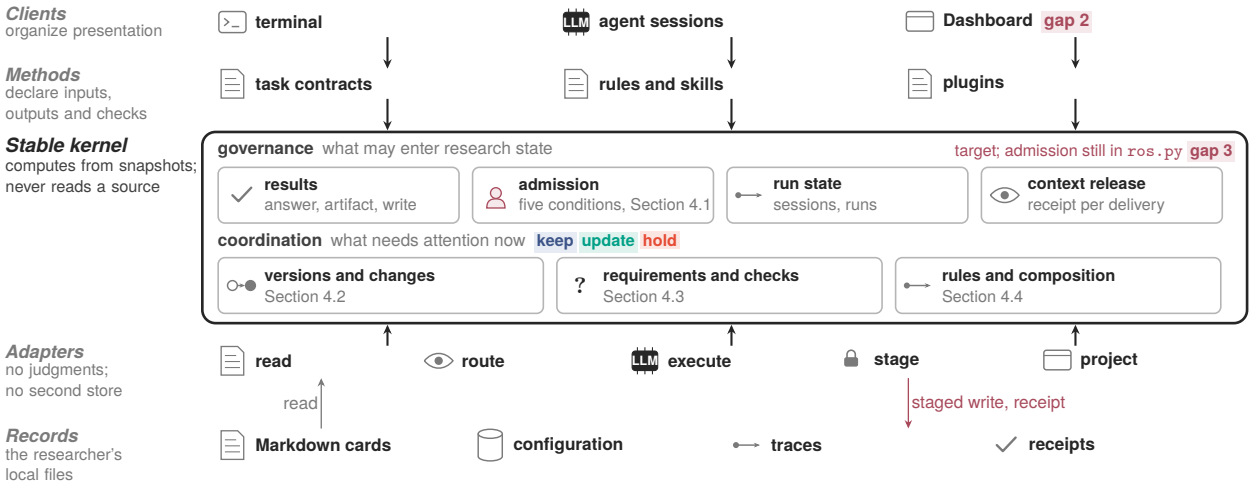


Figure 10. Every dependency points toward a stable kernel that never reads a source directly; the figure draws the target division of responsibilities. Clients and methods depend downward on the kernel, adapters upward. The wine-colored tags mark where the prototype differs: the Dashboard (gap 2), and `ros.py`, which still carries out admission itself (gap 3).

Four structures sit behind the kernel: the record graph, that is, the records and the links between them; the rules; the unresolved requirements; and the execution traces. They are logical structures, not four databases:

Markdown files and explicit decisions remain the researcher's sources, the state of a requirement is derived from the traces that already exist, and caches only speed up recovery. Of these, a trace establishes what was decided, attempted, observed and checked, but it cannot establish a dependency that was never recorded, and it has no authority over the researcher's sources. The prototype has further internal terms, and Supplementary Note 2 gives each with the contract it carries; a new category of term is justified only by a distinct contract or lifecycle. These boundaries are commitments of the design, and whether an implementation keeps to them has to be checked in that implementation.

5.2 Session protocol and admission

Facts and judgments take different routes to research state (Figure 11), which is why the narrowed judgment of Section 2 went through admission instead of being written in directly. A fact may be updated within existing task authority, while a judgment passes through candidate, decision, actual change, check and receipt, each recorded separately. This route is tied to sources from the candidate onward: when `ros collect` or `ros propose` records the candidate, every basis it rests on is frozen at its full source version. If a basis changes during review, admission rejects the stale candidate and requires a new proposal, and an existing judgment can rebind its basis only through an explicit reconfirmation.

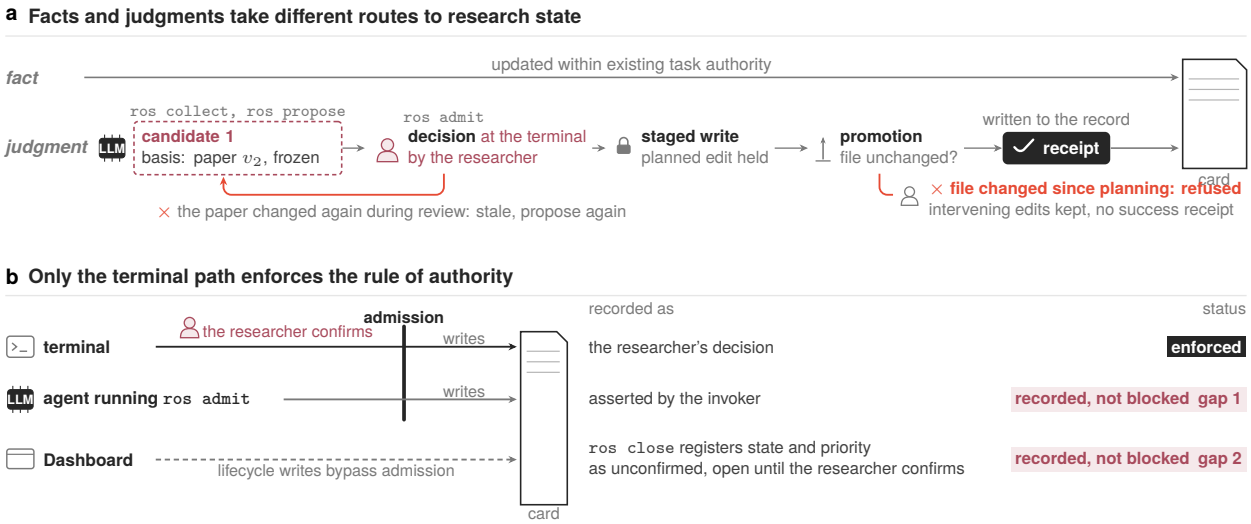


Figure 11. Facts and judgments take different routes to research state, and of the three channels that write a card, only the terminal records the researcher's decision. **a**, The two routes, with candidate 1 of Section 2 as the judgment. **b**, The three channels and how each is recorded (gaps 1, 2).

Similarly, writeback must land on the content it was planned against. If that content has changed by the time the write lands, the write is refused, the intervening edits are kept and no success receipt is issued. This is the refusal described in Section 2, and Supplementary Note 2 gives the mechanism and the paths it covers. These checks protect the built-in writeback paths, but they do not verify what an arbitrary rule must preserve (gap 9).

What an admission records depends on the channel it comes through. A judgment admission confirmed at the terminal is recorded as the researcher's decision. One executed by an agent is recorded as asserted by the invoker and still writes, so for an agent with shell access the record is attribution and an audit trail rather than a block. The record made at the terminal has its own limit: the terminal is recognized by an interactive prompt, which a process that allocates a pseudo-terminal can also answer, and that answer is then recorded as the researcher's decision. The record thus attests the channel, not the person (gap 1).

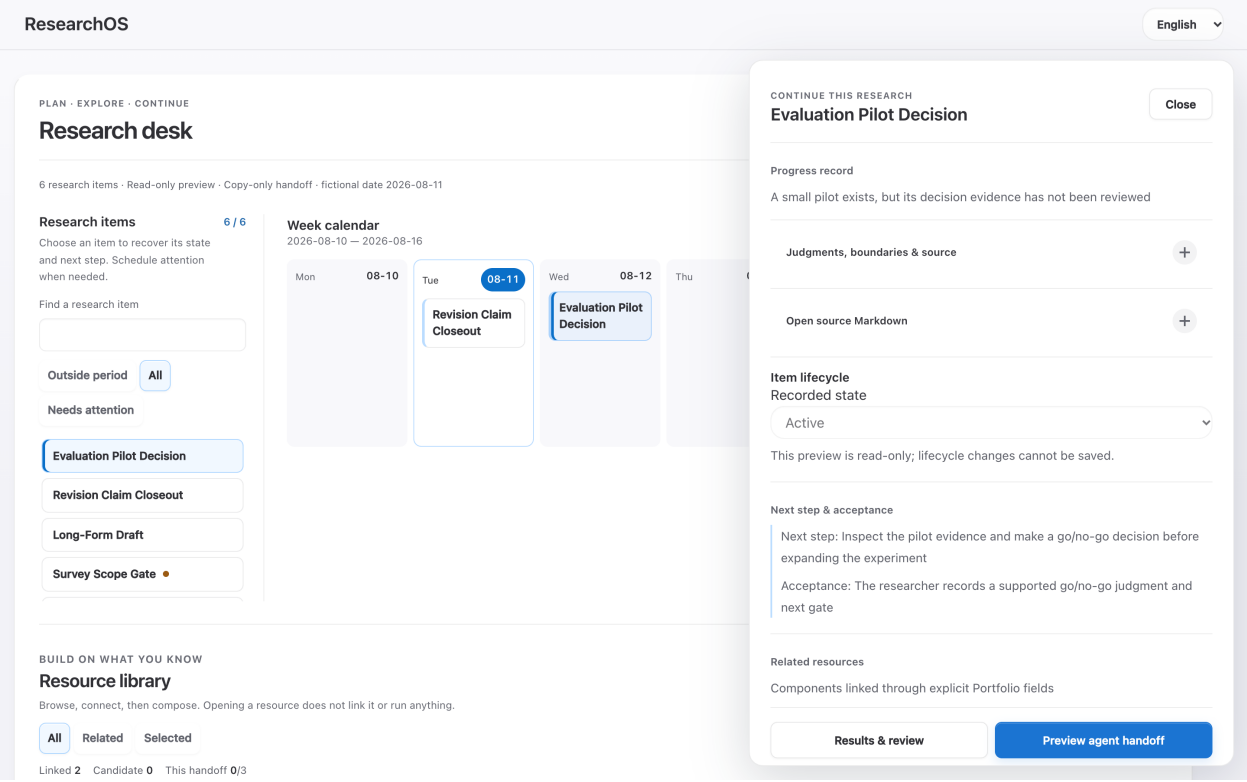


Figure 12. The downloadable version reconstructs an item's research state for the researcher but cannot save it. The Research desk, opened on the fictional item Evaluation Pilot Decision; captured on 29 October 2026 from a fresh clone of the released downloadable version at commit f541cb6, not retouched. The right panel holds the progress record, the judgments, boundaries and source, and the next step with its acceptance criterion.

The third channel is the Dashboard, the prototype's graphical client, whose lifecycle writes bypass admission. When such writes are made during a session, `ros close` registers the fields that carry a judgment (state and priority) as unconfirmed changes that stay unresolved until the researcher confirms them (gap 2). Figure 11b draws the three channels.

Closing a requirement is stricter. `ros decide … cancel/confirm` and `ros answer … cancel/satisfied` are accepted only at the terminal; an agent's relay is recorded as asserted by the invoker, and the requirement stays unresolved. The same rule covers confirming, after the fact, a block written without admission: `ros admit --confirm` run by an agent is recorded as asserted by the invoker, and the confirmation requirement stays unresolved. Closure, on the other hand, does not ask twice: fields carrying a judgment that a complete decision and a successful receipt already cover in the same session are not put to the researcher again, whereas a later direct edit or an incomplete chain of changes still requires confirmation. One admission therefore neither asks the researcher to repeat a decision just made nor licenses arbitrary later changes.

5.3 Clients, methods and trying it

These decisions are taken at the command line, which is also the client that explicitly consumes the kernel's results. The Dashboard's cards, selections and calendars are instead projections, that is, views of the records, so they establish no scientific relation and count as no acceptance. Methods are in a similar position: they declare their inputs, outputs and checks, but declaring them gives a method no authority to accept anything. As for rules, a rule of a kind that already exists is one JSON file, whereas a new kind of executable check needs a change to the kernel and an architecture review, and until then the work it leaves unevaluated blocks completion (gap 12).

Whether each client keeps to these boundaries has to be checked client by client. For example, regression tests run without a graphical interface show neither that graphical interactions pass acceptance nor that external agents are under control, and those agents' hosts keep their own authority. Authority likewise separates the author's prototype from the downloadable version: the two share the same scientific meaning and differ in authority, and a packet that is copied carries no permission with it.

For the Dashboard's home page, the design first considered making the question, grounds, open conditions and next step the main entrance. Because the author preferred a calendar and resource cards for everyday use, the home page keeps that familiar entrance and shows research state in a selected item's details. In choosing resources, selecting one for a handoff only assembles temporary context, whereas recording that it supports a conclusion is an assertion to be admitted, so the two should not share one drag action. Which arrangement other researchers would find better is not yet known, and neither the author's preference nor the implementation checks can show it.

Trying it. The downloadable version is copy-only by default. Its shipped `config/ros.json` sets `runtime_scope: public_starter`, and the behavior below holds under that setting. On its fictional fixture `examples/scheduler-demo`, a first session takes the steps of Section 2 with the commands in the right-hand column of Table 2. Because the downloadable version dispatches to no AI model provider and admits nothing into the researcher's sources, the records that count as authoritative, a user can try recovery, context delivery and candidate collection, but not admission.

Figure 12 shows the downloadable version's Research desk, which cannot save an item's lifecycle, and Figure 13 its agent handoff, which it can only preview and copy. The copied packet opens with its digest and copy_only route and continues with the goal, the session sources and the context route. The two screenshots show how it presents an item; they do not show task completion, accepted judgment or usability.

Compared with the downloadable version, the author's prototype adds admission and Dashboard writeback. Its session kernel can also hand a bounded run to a local runner, the Codex CLI or the Claude Code CLI, each only when declared dispatchable under `providers`. Each starts in an empty temporary workspace with its settings and access restricted, as Supplementary Note 2 details. The downloadable version and the author's prototype also use different Python versions: the former in CI on CPython 3.10 and 3.13, the latter on CPython 3.11 (Appendix B.3).

The author's prototype runs on the author's own records and is not distributed; a user who wants to work on personal Markdown notes can start the downloadable version in its personal-vault mode. This mode builds a local Dashboard view from those notes and, after the user confirms, writes Dashboard changes under `personal_vault/`; if configured, it can also start a locally installed Codex CLI. However, `ros` itself does not change in that mode, and `ros admit` still only shows the planned difference. Nor does `ros` intercept ordinary agent sessions, which continue as before; only judgment blocks that come back through

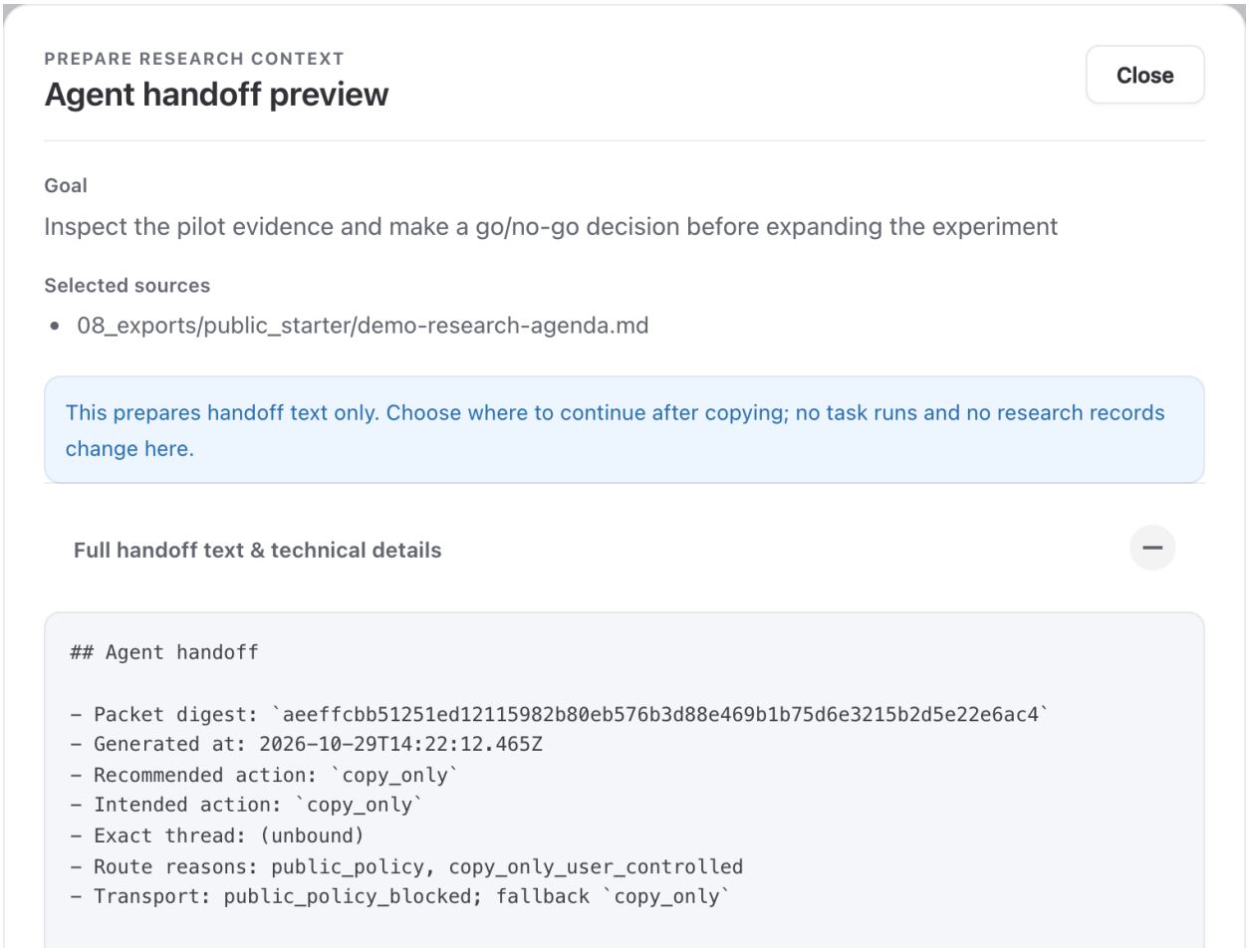


Figure 13. The handoff is text the user copies; nothing runs and no record changes. The agent handoff of the downloadable version, previewed for the item of Figure 12; captured on 29 October 2026 from a fresh clone of the released downloadable version at commit f541cb6, cropped from the top of the dialog to the head of the packet, not otherwise retouched. Above the notice: the goal and the selected source.

`ros collect` or are registered with `ros propose` enter the candidate path. Setup time and per-session effort have not been measured.

5.4 What the prototype does not yet do

The session of Section 2 differs from the definitions of Section 4 in one place, and the difference runs through the whole session. By the definitions, the recheck of J is held, with a question to the researcher (Section 4.3 and Figure 7); in the session it is unresolved but not held. The difference comes from what the prototype can check. On this recheck, the only check it can decide by itself is whether the basis is pinned to the current version, and after the correction that check fails (returns 0). A failed check is not one that returns cannot yet tell, and only the latter is held, so there is nothing to hold and the recheck stays plainly unresolved. Whether the first-set result still supports J is left to the researcher (gap 34, semantic checks).

By contrast, the prototype holds a requirement only when it meets a check it cannot decide and must put to the researcher: `ros today` asks once, and the requirement stays held until the researcher answers with `ros answer`. However, no rule shipped with the prototype declares such a check, one that rests on the researcher's

judgment. The held path is therefore reached in the session kernel's regression tests and otherwise only when a requirement's subject or basis no longer resolves (Appendix B.1).

This difference is one case of the general relation between the definitions and the code. Section 4, made precise in Supplementary Note 5, is the specification; the current code supplies selected executable paths, and coverage is recorded per obligation rather than inferred from test counts. Table 4 groups the most important gaps by theme, and Supplementary Note 6 gives all 39 in full, under the numbers used throughout this report. The gaps bound what the implementation can claim and leave the definitions of Section 4 and their conditional arguments unchanged; similarly, the architecture of Section 5.1 assigns responsibilities rather than claiming that every composition or client exists.

Three gaps matter most to a user of the prototype. The first is that only the terminal path enforces the rule of authority: Dashboard writes and agent-executed admissions are recorded, as Section 5.2 describes, but not blocked (gaps 1 and 2). The second is that a source change is noticed only at the next `ros` command, because no file watcher exists, and it does not propagate across several hops (gap 34). The third concerns waiting: a deferral the researcher confirms at the terminal with `ros decide … defer` does produce waiting (Appendix B.2), but no mechanism yet produces waiting on an out-of-scope dependency (gap 17).

The prototype departs from the definitions in several further places. Besides leaving to the candidate which part a correction touches (Section 2, gap 11), it cannot replace or withdraw J; the researcher edits or removes it by hand. By the definitions, the unknown under the re-appearance condition stays among the open conditions and the experiment remains an unresolved requirement. However, the prototype has no rule to register a stated condition or a planned experiment, so `ros close` lists neither (gap 12). The experiment would be waiting only if the researcher deferred it at the terminal with `ros decide … defer`. The session makes no such deferral, so the experiment stays unresolved and the goal is not Done.

The prototype also identifies a version by its content digest alone, so a return to earlier content counts as the earlier version rather than as a new revision (Appendix B.1). Finally, it does not implement complete conditional composition (gap 10), sessions only approximate bounded goals (gap 15), and cross-client integration remains unverified.

Two kinds of evidence have to be kept apart in assessing this prototype. The regression tests of the kernel and of writeback exercise source changes, proposal freezing, admission, refusal of concurrent writes and closure records on temporary fixtures, and passing them establishes those execution properties for those inputs. Whether the judgments for the in-view and re-appearance conditions are right, whether the experiment is worth running and whether the researcher understood more are questions for actual tasks and independent evidence.

6 Neighboring systems, the strongest counter-case and conclusion

Closing these gaps would leave two questions. One is whether the tools researchers already use already do what ReSearchOS sets out to do. The other is whether it would be enough to write the state of the work into a one-page versioned handoff document and give it to a strong model that can read and write files. Setting ReSearchOS beside neighboring systems, class by class, answers the first; only the comparison described in Section 6.2 can answer the second.

Table 4. Disclosed implementation gaps grouped by theme

Theme	Gaps (numbers in Supplementary Note 6)	What it means for a user of the prototype
Admission and writeback	1, 2, 9	Agent-executed admissions and Dashboard writes are recorded but not blocked, and the built-in checks do not verify what an arbitrary rule must preserve.
Bounded goals and completion	15, 17, 18	Sessions only approximate bounded goals: specification freezing is incomplete, waiting on an out-of-scope dependency is not produced, and a held question on an obsolete basis can still let a session report complete_with_held.
Composition and maintenance	3, 10, 11, 12, 34	The single stable kernel remains a target because `ros.py` still carries out admission itself. General composition checks, protection of independently supported parts during a correction, registration of stated conditions and planned experiments, file watching and propagation across several hops remain specified only.
Acceptance and benefit	13, 14	Human acceptance of the interface, isolation of real models, long-run recovery and research benefit are unverified.

6.1 Neighboring systems

The nearest alternatives to ReSearchOS are not research agents but the tools researchers already use. To compare against those tools, the paper of the running example is corrected once in each class of tool. Table 5 records, class by class, what is kept, what is updated when a source changes and what can stay unresolved; a row describes the class as its named or typical systems report it, not every system in it. Figure 14 shows how the correction is handled in six of the classes, and none of the six leaves the change unresolved as a question for the researcher, as the last column of Table 5 also records.

In a note tool the corrected paper changes nothing: the note that records J stays as it was until the researcher edits it, by hand or through an assistant. Version control keeps every version of the paper's record, merges a concurrent branch and refuses a stale write. It refuses by optimistic concurrency control [32], also the mechanism behind the prototype's refusal of a changed file, and it detects concurrent branches as version vectors do [33]. Version control does not, however, distinguish a fact from a judgment, and its answer to a conflict is to merge. ReSearchOS instead keeps conflicting branches as current version candidates (Supplementary Note 5) and leaves the choice to the researcher.

Several classes, by contrast, resolve the correction themselves. In a truth maintenance system, with the two benchmark sets drawn as separate justifications, the one through the second set fails and J stays in through the first (Figure 14c). Such a system [34] records why each belief stands and revises beliefs along those recorded justifications: once the reason for retracting an assumption fails, the assumption is restored, and a belief that still has another valid justification stays in. The logic-based TMS [35] labels each belief true, false or unknown, and the assumption-based TMS [36] keeps every consistent context at once, without retraction.

The checks of Section 4.3 use the same three values, and the touched set of Section 4.2 is simpler, marking requirements only one step along recorded dependencies. What separates a truth maintenance system from

ReSearchOS is who gives the value. In a truth maintenance system the system relabels, and unknown means that propagation derived neither true nor false. In ReSearchOS, a check that the system cannot decide takes its value from the researcher, within a frozen scope, so a change can leave the recheck of a judgment held for the researcher. Belief revision in the AGM tradition [37] defines how a theory changes through functions governed by a set of postulates, whereas keep, update and hold name what happens to the researcher's records and are not revision operators.

A build system rebuilds the result with the person outside the loop. Build systems [38] decide what a changed input invalidates, and those with early cutoff stop when a rebuilt value is unchanged, which is the update operation for artifacts. A suspended build task, however, waits for its dependencies, not for a person's judgment.

Agent memory keeps and retrieves, and when the paper changes it overwrites or invalidates the old chunk as the agent decides. Graphiti [39] invalidates a contradicted fact instead of deleting it, GitHub Copilot Memory [40] checks the code locations a memory cites against the current branch before use, and PACMI [41] marks the memories that depend on changed evidence as needing verification. In all three, the system sets the new status and no person is asked. GovMem [42] routes a doubtful candidate memory to human review, but only before it is written, and describes no later recheck of a memory once it is promoted.

An AI-scientist system reruns, and keeps, updates and produces artifacts, under its own authority. Agent Laboratory [43] accepts a researcher's feedback at each stage, and Kosmos [44] cites every statement of its reports to code or literature within a run of up to twelve hours. Neither these systems nor the build systems and agent memory above record an admitted, scoped judgment that a later source change reopens.

Research knowledge graphs and experiment trackers update the record of the paper, not the grounds of a judgment built on it. A research knowledge graph [45] would make the paper and its comparison a first-class object, with versions and provenance for the stored data; the judgment protocol here adds the scope, checks and authority that decide whether a judgment built on it still holds. Experiment trackers and provenance stores keep runs and lineage; they answer what was run, not whether a judgment still holds.

Other tools bring the change back to a person, a practice older than this report. Requirements tools such as IBM Engineering Lifecycle Management mark a trace link suspect when the content of either linked artifact changes, and a team member then marks it valid or invalid [46]. On GitHub, a protected branch can be set to dismiss an approval once new commits change the diff it approved, and the change cannot merge until someone approves it again [47].

Living systematic reviews bring research conclusions back to their authors in a similar way, usually when new evidence is found. The authors judge whether new studies are likely to have an important impact on the review's findings, and readers are told that an update is pending while the new evidence is integrated [48]. In June 2026 Cochrane announced that it matches retracted studies to the reviews that include them, and that it will contact the authors, add editorial notes to affected reviews while the impact is assessed, and leave the decision on the conclusions to its editors [49].

Closer to a single claim, a ledger of the quantitative claims in one paper [50] fingerprints the data behind each claim. It flags a claim as stale when that data changes even if the claim still passes, and it blocks the claims that depend on one that fails.

One recent system carries a retraction down a whole chain of records. In a wiki maintained by a language model for regulated settings, Zhang's provenance layer [51] binds claims to evidence passages. When a source is marked retracted, superseded or downgraded, it follows those links to the dependent claims, rules and decisions. A claim with other support has its confidence recomputed and is marked disputed below

a threshold, a claim with none is suspended, and the decisions downstream are flagged; a decision whose confidence falls below a floor is flagged for human review. Five simulated retractions on a 112-document corpus reached every affected record.

This is the nearest mechanism this report found to the touched set of Section 4.2, and it reports a controlled evaluation, which ReSearchOS does not. The difference lies in who acts and on what: the system sets the status of a whole claim by threshold, and the paper describes no scope on a claim and no question to a person about which part still stands.

What these leave unresolved is a link, a pending change, a whole review or a whole claim. None keeps admitted research judgments with their scope and basis version, updates only the part a change touches through a candidate the researcher admits, and holds a check with a question for the researcher. The comparison also cuts the other way: a new study can make a judgment unreliable without changing any input the judgment depends on (Section 4.2). Living reviews are built for that case, and ReSearchOS does not notice it. The ReSearchOS row of Table 5 describes the design mechanisms, and Section 5.4 states how much of them is executable.

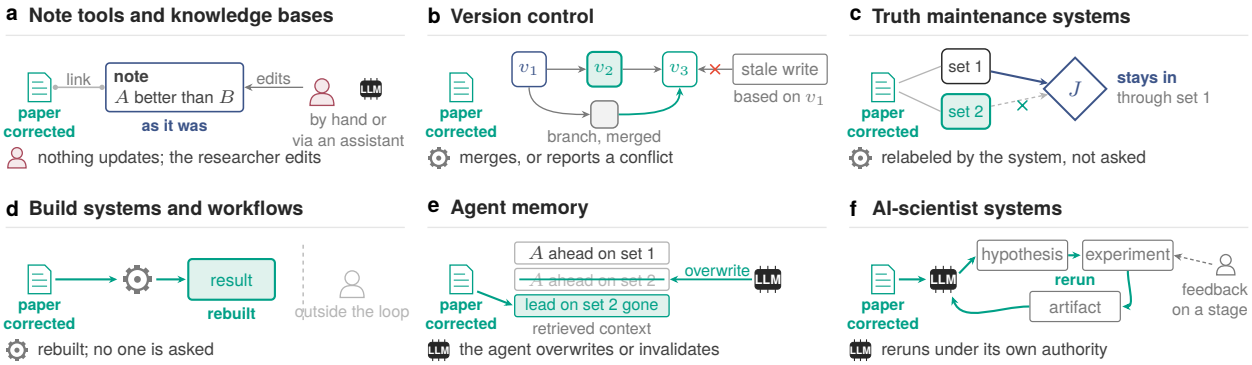


Figure 14. When the paper is corrected, each neighboring class of system resolves the change itself or leaves it to the researcher to edit. a–f, The same correction, at the left of each panel, in six classes of Table 5, with the colors of Figure 1. The foot line of each panel names who resolves the change.

Table 5. Neighboring systems by the three operations

System class	Keeps	Updates when a source changes	Leaves unresolved
Personal knowledge bases and note tools	Notes and links	Nothing when a source changes; the researcher edits, by hand or through an assistant	Nothing; an unresolved question is a note
Version control with optimistic concurrency	Every version; refuses a stale write	Merges, or reports a conflict	Branches, without saying what they mean
Truth maintenance systems (JTMS, LTMS, ATMS)	Conclusions with their justifications	Relabels along justifications on its own; a conclusion with another valid justification stays in	Unknown labels (LTMS) or several consistent contexts at once (ATMS), maintained by the system, not put to the researcher

System class	Keeps	Updates when a source changes	Leaves unresolved
Retraction services and claim ledgers	Reviews and their studies, or claims with a fingerprint of their evidence or bound to evidence passages	Flags a retracted study, marks a claim stale when its data change, or marks claims disputed or suspended by threshold and flags the decisions downstream	A whole review, to which Cochrane announced it will add an editorial note, a whole claim until its author updates it, or a whole claim the system suspends; nothing scoped, no candidate to admit
Build systems and scientific workflows	Keys, values and their inputs	Rebuilds what the change invalidates	Nothing; the human is outside the loop
Research knowledge graphs and experiment trackers	Comparison records, runs and lineage	The record, not the judgment's grounds	Nothing
Agent memory (RAG, A-MEM, Mem0, MemOS, Graphiti, PACMI)	Retrieved and compressed context	Overwrites, re-ranks, or invalidates as the agent decides	At most a needs-verification mark set by the system; nothing put to the researcher
AI-scientist systems (AI Scientist, Agent Laboratory, Co-Scientist, Kosmos, Denario)	Hypotheses, experiments and artifacts	Reruns under their own authority; a researcher may give feedback on a stage	Nothing the researcher decides
ReSearchOS	Judgments with scope and basis version	Only the touched part, through a candidate the researcher admits	Unresolved scientific conditions, and checks with a question for the researcher

Researcher authority itself has precedents, and what matters in each is what happens to an approval when the paper is corrected. OmniScientist's OSP protocol [52] records a human's approval of an artifact, with its version hash, in a provenance chain kept on a central hub. What a human approves there is an artifact that an agent submits for review, not a scoped judgment that a later source change reopens. In the running example, approving only the ranking of A over B and passing it to the next task would conceal the untested re-appearance condition.

Two recent systems come closer on researcher authority. Magnolia [53] applies proposed cross-project rules only after a researcher confirms each one, and ResearchLoop [54] admits a claim only within the scope that a gate, written by the researcher or a prespecified protocol file, approves. Neither paper, however, describes returning an admitted result to the researcher when one of its sources later changes. In Magnolia, a rule tied to a superseded tool version stays until the researcher revises or retires it; in ResearchLoop, the recheck is a deterministic gate evaluation that downgrades or blocks a claim whose evidence chain is missing or mismatched.

Figure 15 places most classes of Table 5, and several systems named in this section, on two axes. One is what a source change leaves unresolved, taken from the last column of Table 5 or from the paragraph that describes the system. The other is who acts on the change: the system or an agent, which relabels, merges, rebuilds, reruns or overwrites; a person, who edits, re-approves or updates the whole item directly; or the researcher, who admits a candidate for the touched part of a judgment. Research knowledge graphs and experiment trackers are not placed, because Table 5 does not say who updates their record. Only ReSearchOS leaves the touched part unresolved and settles it through admission.

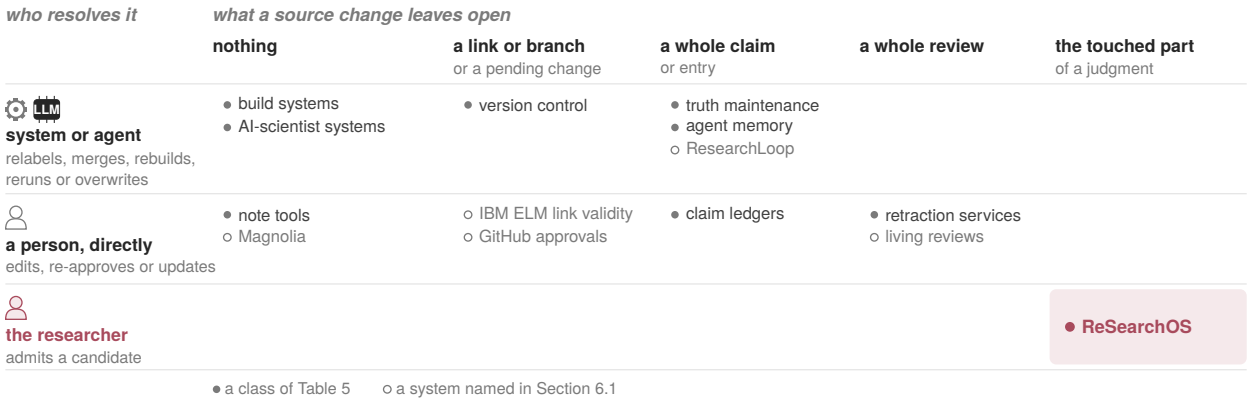


Figure 15. Neighboring systems differ from ReSearchOS in what a source change leaves unresolved and in who resolves it. Filled dots are classes of Table 5, with retraction services and claim ledgers, one row of the table, drawn apart; hollow dots are systems named in Section 6.1.

The comparisons above record mechanisms as their sources report them, not integration or performance; Supplementary Note 4 gives the full synthesis, including the retrieval, memory and AI-scientist systems behind Table 5 and other work that supplies parts of the design. One point from Section 3.4 also carries over: accumulation must accommodate correction and must not be confused with human learning. Studies of generative AI and learning [17], of human–AI combinations [20] and of feedback loops [19] motivate evaluating three things separately: artifacts, strategy reuse and the researcher's learning. Whether the environment is needed at all, however, lies beyond these comparisons, and that is the question of the counter-case.

6.2 The strongest counter-case and the smallest form

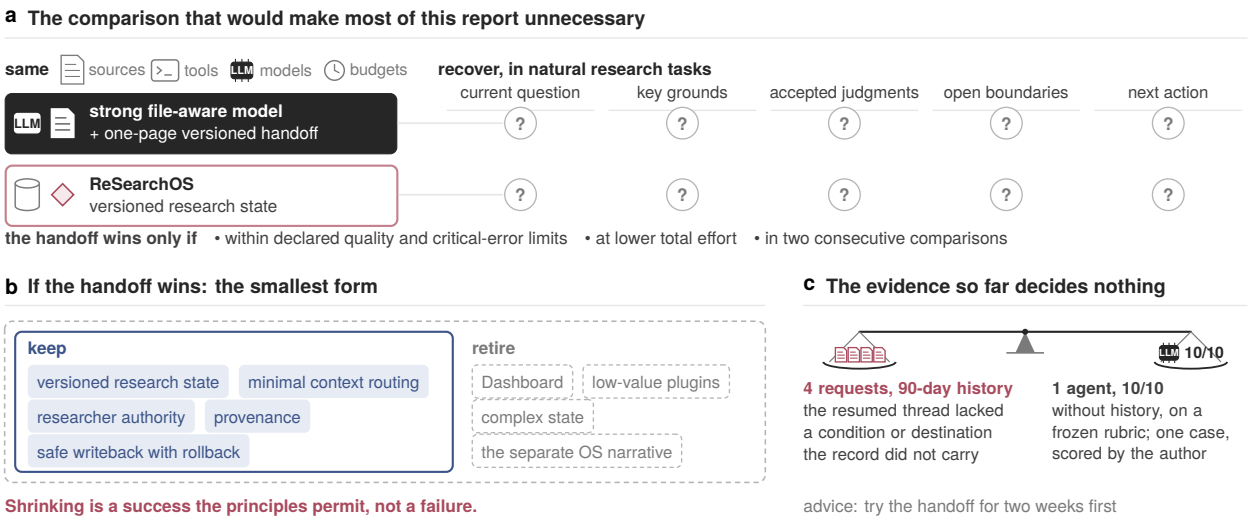


Figure 16. One comparison would make most of this report unnecessary, and the evidence so far decides nothing. **a**, The comparison, not yet run, so no cell is scored. **b**, The smallest form (blue frame) if the handoff wins, and the four items that could then be retired. **c**, The evidence so far on each side.

One result would make most of this report unnecessary, and the design documents behind Section 3 name the comparison that could produce it: a strong model that can read and write files, given a one-page versioned handoff document, set against ReSearchOS over enough natural research tasks with equal sources, tools, models and budgets. The handoff wins if the model recovers the current question, the key grounds, the accepted judgments, the unresolved boundaries and the next action within declared quality and critical-error

limits, at lower total effort, and in two consecutive comparisons. In that case ReSearchOS should shrink to versioned research state, minimal context routing, researcher authority, provenance, and safe writeback with rollback, and the Dashboard, low-value plugins, complex state and the separate operating-system narrative could then be retired. Such shrinking is a success the principles permit, not a failure (Figure 16).

The evidence so far points both ways and decides nothing. On one side are four requests from the author's reconstructed ninety-day design history, listed in Table 6 and drawn in Figure 17 in the style of Figure 1. In each, a resumed thread is known to have lacked a condition or a destination that the record of the previous session did not carry. They are the only such cases so far, and what was missing in them is what a one-page handoff would have to supply. Since none of them is a source change, this report suggests that the natural tasks of the comparison include one in which a source changes, as the paper does in the running example.

Table 6. Four historical requests and what resuming lacked

Request	What resuming lacked	Design response
Combine two collaborator cards; the task appeared complete	The output, and where a change belonged, were not visible	Return an answer with its grounds and name each proposed destination separately; a source can be read without being edited
Borrow from an earlier memory-related mechanism for a design note, without forcing the old structure onto the new task	What problem the mechanism solved, and which changed assumption could defeat the transfer	State the correspondence, the changed assumptions and the smallest check that tells them apart; the candidate may be rejected
Connect a new application interest, and two potential contributors' experience, to existing work	Access, time and willingness, with no question or team yet fixed	Keep these as feasibility conditions of a candidate question; experience explains a suggested role but is not consent
Attach the understanding of two plan documents for a project the author supports, and find it later	The source locations, the role boundary and what remained unresolved	Link the documents and the role to a locatable brief; frequent activity does not turn support into ownership

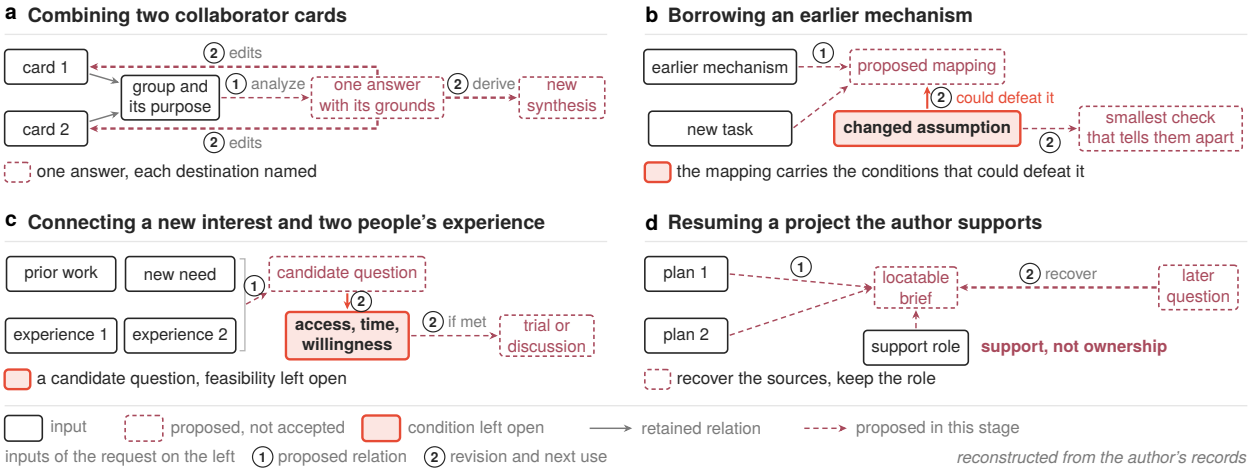


Figure 17. In each of the four requests, what resuming lacked was a condition or a destination that the record did not carry. **a**–**d**, The requests of Table 6, inputs at the left. Mark 1 is the proposed relation, mark 2 the revision and next use. Foot lines give the design response. Dashed objects are proposed; red-orange boxes are unresolved conditions.

The rows of Table 6 paraphrase the author's own records and are the author's retrospective reconstructions. They serve as precedents for where the design needs came from, not as recovered execution traces or mea-

surements of benefit, and they do not show that a maintained handoff would have failed on them. The same history records repeated requests for a less cluttered interface, and the design response recorded there is to run the same session with a strong model that can read and write files and a maintained handoff as the comparison before expanding the interface.

On the other side, one history-free agent given a participant package reached 10 of 10 on a frozen rubric without hard failure (Appendix B.1). That single case, scored by the author, does not establish that the handoff alone is sufficient, that it is reliable across tasks or models, or that it has value for participants.

The next step is the coordination policy, the part of the environment that decides, at each decision boundary, what to check, start, ask or leave alone, together with a task that tests whether a policy keeps the right research state over time. Neither the comparison with the handoff baseline (a strong model with a one-page versioned handoff document set against ReSearchOS under equal conditions) nor the first study of use, with researchers other than the author, has been run, and nothing in Sections 2–5 presumes their outcome. Until they have been run, the practical advice is to start with the handoff: write the state of the work into a one-page versioned document, give it to a strong model that can read and write files, and use it for two weeks of real work, installing the downloadable version only where this loses a condition or a destination. The downloadable version collects candidates but does not admit them (Section 5.3).

6.3 Conclusion

ReSearchOS understands continuity in research as the continuation of research state rather than of files. This understanding is worked out in three parts that constrain each other: the principles say what must survive a change, the definitions say what research state is and what the three operations on it require, and a reference architecture and prototype assign those operations to records, a kernel and clients. In the worked session, the prototype recovered the comparison, marked the judgment for recheck when its source changed and admitted the narrowed judgment only on the researcher's decision; it also recorded the researcher's refusal of the unsupported proposal, and the recheck was still unresolved at closure. These are starting paths, checkable at the terminal.

To separate the evidence given from what remains to be shown, Table 7 sets each contribution beside both.

Table 7. Contributions, evidence and what remains

Contribution	Evidence given in this report	What remains to be shown
Design principles (Section 3)	The four requests of Table 6, as precedents for where the design needs came from, and the literature that motivates each principle	Whether they improve understanding and decisions in actual research; no study of use has been run
Definitions of research state (Section 4)	The definitions, and two propositions argued under their stated hypotheses (Supplementary Note 5)	The prototype does not implement complete conditional composition; neither proposition shows that completion is reachable
Reference architecture and prototype (Section 5)	The worked session (Section 2), kernel regression tests and the CI runs of Appendix B.3	Not implemented: registration of stated conditions and planned experiments, and waiting on an out-of-scope dependency. Unverified: cross-client integration. Not run: the comparison with a strong model that can read and write files, given a one-page versioned handoff document (Section 6.2)

Long-term benefit has not been shown. Neither of the two studies of Section 6.2 has been run: one sets a strong model with a one-page versioned handoff document against ReSearchOS under equal conditions, and the other is a study of use with researchers other than the author. The results of each should revise the definitions or the prototype before any value is claimed. Where a clear handoff is sufficient, the simpler arrangement should be preferred; additional structure deserves to remain only where it helps researchers notice changed conditions, revise judgments and avoid repeated work. This returns to the answer the report began with: what has to be kept is research state, not research files, and when a source changes, the researcher, not the AI model or another session, decides which part is kept, updated or left unresolved.

Availability and ethics. The reference artifact uses public literature and fictional examples. The four design requests mentioned in Section 6.2 were reconstructed from a ninety-day history of the author's own records, which contains messages from collaborators and students and unpublished work. That history was processed only locally, its symbolic projection carries no identifiers, and nothing from it is distributed. Personal and project identifiers, original private documents, unpublished technical details, private sources and record graphs are excluded from the public report, and this report does not announce a dataset or benchmark release. The current software license is noncommercial source-available, without an OSI-approved open-source claim. Artifact access remains subject to repository visibility. Any study with participants requires consent, applicable institutional review, data minimization and withdrawal arrangements, and safety experiments require isolation from real external effects.

Disclosure. AI assistance supported source discovery, drafting, translation, figures and typesetting; the author is responsible for the scientific claims and submission metadata. The bibliography includes relevant coauthored studies as methodological sources, without treating their results as validation of ReSearchOS. No participant outcomes or long-term causal benefits are reported.

A Objects and the lifecycle of a requirement

Section 4 introduces these objects one at a time, on the example; this appendix puts them side by side. Table 8 lists each kind of object the definitions cover, the states it can be in, the condition under which it moves and the party or mechanism that moves it. The parties are the researcher, the system (the kernel and the commands of Sections 2 and 5) and an AI model, whose output enters only as a candidate. Supplementary Note 5 states each of these definitions precisely and argues the two propositions of Section 4.4.

Table 8. Objects, states and who changes them

Object	States	Condition	Moved by
Source or record version	Current version candidate, or superseded	Current while it is a recorded revision with no known successor among the permitted observations available by the cutoff; a successor is known only when the visible history establishes it, so several unresolved branches can be current at once	The system, at each decision boundary; in the prototype, at the next `ros` command
Candidate	Collected and undecided, admitted, or refused	Admitted only when the five conditions of admission hold (Section 4.1), beginning with the researcher's decision; refused as stale when a basis changed during review. A file changed since the write was planned refuses the write, not the candidate	`ros collect` or `ros propose` collects it and freezes its bases; the researcher admits or refuses it; admission itself refuses it as stale
Judgment	Kept, marked for recheck, or updated	Kept while a change touches none of its grounds; marked for recheck when a requirement on it is touched; updated only when a candidate that narrows or replaces it, pinned to the changed source version, is admitted	The system marks it; the researcher's admission updates it. In the prototype the update is a new block beside the old judgment, which the prototype can neither replace nor withdraw; the researcher edits or removes it by hand (gap 11)
Requirement (a piece of maintenance work)	Unresolved, resolved or cancelled; held and waiting are kinds of unresolved work	Unresolved once a change touches one of its recorded inputs or a scan finds it. Resolved when every check is 1 on one eligible bundle of receipts on current versions; cancelled by a recorded decision. A later change can touch it again	The system registers and resolves it; the researcher cancels, defers or answers
Held requirement	Held on a question	It has a trigger that reopens it, at least one check is ?, and every check that is not 1 is a ? that only the researcher can remove, each with its own live question	The researcher's answer or the trigger: an answer that establishes satisfaction closes the check, one that establishes non-satisfaction returns the work to the unresolved state, and the trigger reopens it
Waiting requirement	Waiting on a trigger	Its last recorded transition is a wait or a deferral, it has a trigger, and a dependency outside the frozen scope blocks it or the researcher deferred or narrowed it	The researcher's deferral or narrowing, or an out-of-scope block, puts it there; the trigger reopens it

Object	States	Condition	Moved by
Check	1 (pass), 0 (fail) or ? (cannot yet tell); a check never evaluated has no value	1 or 0 only when a qualified, currently valid check establishes satisfaction or non-satisfaction; ? when the evidence is missing, stale or in unresolved conflict; a check with no value is unevaluated work that blocks completion	A mechanism, for a check whose basis is mechanical; the researcher's decision and reason, for a check whose basis is human
Bounded question	Pending, asked or answered; invalidated by a version change	Asked once for each version of the check's basis, with a finite set of options; live while it is pending or asked by the cutoff; a question invalidated by a version change has not been answered	The system asks; the researcher answers
Bounded goal (a session in the prototype)	Running; stopped with outcome `complete`, `complete_with_held`, `deferred`, `budget_stopped` or `human_stopped`	Specification frozen at the start and admitted like any candidate; stops at Done, at an exhausted budget component or at the researcher's decision to stop; the outcome is read off at the first Stop	The system freezes the specification (`ros today` in the prototype) and evaluates Done and exhaustion; the researcher may stop it

Of these objects it is the requirement that carries the maintenance work, and Figure 18 draws its lifecycle. A requirement is touched when one of its recorded inputs changes, compared by revision digest, or when a scan finds it as work is set up or a scoped check is rerun. It closes only when every check is 1 on one bundle of receipts bound to current versions, or when a recorded decision with a reason cancels it. Until then, an answer of 0 or the trigger returns held work to plain unresolved work, and the trigger reopens waiting work.

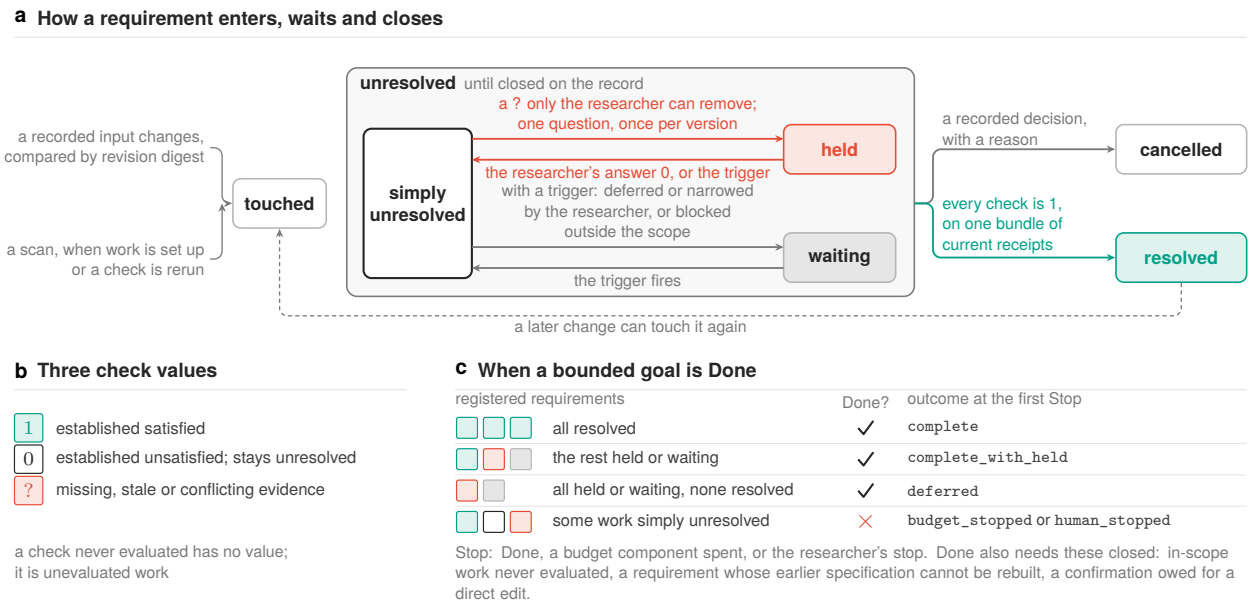


Figure 18. A requirement stays unresolved until it is closed on the record; held and waiting work is still unresolved. a, How a requirement enters, waits and closes. **b**, The three check values; a check never evaluated has none. **c**, When a bounded goal is Done, and the outcome read off at the first Stop.

Holding and waiting keep a requirement in view without closing it, since only resolution and cancellation remove it from the unresolved set. A 0 is established non-satisfaction and keeps the work unresolved; a check never evaluated has no value and counts as unevaluated work. A bounded goal is Done when every unresolved registered requirement is held or waiting and no unevaluated work, unreconstructable specification or owed confirmation remains unresolved. A goal with held or waiting work can therefore still be Done, as complete_with_held, or as deferred if none of its requirements is resolved.

B Implementation and engineering record

Sections 2 and 5 say what the prototype does; this appendix records the commits, commands and tests it rests on. B.1 maps each definition behind Section 4 to the commands and regressions that carry it, B.2 says how a session stands in for a bounded goal, and B.3 gives the commits, the same as in Sections 2 and 5, and their tests; Supplementary Note 6 lists every disclosed gap. Design obligations, formal definitions, implemented paths and evidence are kept as separate claims, so no unimplemented mechanism is presented here as a runtime capability. Implementation statements not made for these commits keep the scope in which they were validated earlier and were not accepted anew at these commits. Components kept in the repositories but not under development, such as the plugin system, are described as they stand, and this report adds no functionality to them.

B.1 From definitions to commands

Table 9 answers three questions for each definition behind Section 4: through which commands it reaches the researcher, which kernel regressions in `evals/test_ros_kernel.py` exercise it, and what the prototype still lacks. The numbered definitions are in Supplementary Note 5.

Table 9. Definitions of Section 4 in the prototype

Definition in Section 4	Commands	Kernel regressions	What the prototype still lacks
Research state (Section 4.1)	`ros state`, `ros context`	`test_state_reports_q_e_j_b_n`; `test_cr2_ros_context_emits_context_receipt_with_egress_service`	Stated conditions and planned experiments are not registered (gap 12); agents' direct reads lack receipts (gap 7)
Admission (Section 4.1)	`ros collect`, `ros propose`, `ros admit`	`test_hj1_ros_admission_requires_human_decision_record`; `test_proposal_freezes_unpinned_basis_and_admit_rejects_later_change`; `test_admission_does_not_overwrite_an_edit_after_planning`	`test_hj1_…` checks that the channel is recorded, not that the researcher's decision is required; agent-executed admissions and Dashboard writes are recorded, not blocked (gaps 1, 2); what an arbitrary rule must preserve is not verified (gap 9)

Definition in Section 4	Commands	Kernel regressions	What the prototype still lacks
Current version candidates (Definition 5.1)	`ros today`	`test_cr16_exact_replay_adds_no_observation_work_or_question`; `test_today_explains_exact_changes_and_related_requirements`	Versions are content digests, one per record at each boundary, so the order, branches and late ancestors of Definition 5.1 are not computed. The cited regressions test replay and change detection only; a change is seen only at the next `ros` command, with no file watcher (gap 34)
Touched and unresolved requirements (Definitions 5.2–5.3)	`ros today`	`test_basis_change_opens_then_reconfirm_resolves_and_replay_is_deterministic`; `test_a_b_a_reopens_and_generation_advances_on_new_read_set`	A return to earlier content counts as the earlier version and resolves the recheck, unlike the new revision of Definition 5.1; no propagation across several hops (gap 34); attribution of new requirements is partial (gaps 23–25)
Three-valued checks and resolution (Definitions 5.4–5.5)	`ros today`; `ros decide … cancel` at the terminal	`test_a_requirement_closes_on_its_declared_check_bound_to_the_current_version`; `test_cc3_unknown_check_never_resolves_and_stays_apart_from_coverage_unknown`; `test_b8_only_the_terminal_closes_a_requirement_by_decide`	Only script checks run, and a new kind of executable check needs a kernel change (gaps 12, 25)
Held and waiting work (Definitions 5.6–5.10)	`ros today`, `ros answer`, `ros decide … defer`	`test_hj6_undecidable_unknown_is_asked_once_and_waits_outside_open_work`; `test_cc7_defer_channel_controls_completion_and_terminal_decline_does_not_write`	No shipped rule declares a human-basis check, so outside the kernel regressions the held path is reached only when a requirement's subject or basis no longer resolves. No producer for waiting on an out-of-scope dependency (gap 17); holds on obsolete questions can survive (gap 18)
Composition (Definition 5.11)	None	None	No execution consumer for general successor, parallel or join checks (gap 10)
Bounded goal, Done and Stop (Definition 5.20)	`ros today` (also with `--for`), `ros close`	`test_cc6_episode_spec_is_frozen_and_scan_stays_inside_it`; `test_cc7_special_requirements_exclude_done_in_every_unclosed_state`; `test_cc10_waiting_spends_boundaries_and_an_exhausted_session_stops`	Sessions only approximate bounded goals (gaps 15, 16); budgets are not fully metered (gaps 19–22); the zero-share rule is not enforced (gap 18)

Each row reads from a definition to the command a researcher types, the regression that runs that command on a temporary fixture, and the gaps that bound the claim. The regressions establish the named execution

properties for their fixtures; whether a definition is implemented in full, whether a judgment is right and whether the researcher gained anything lie beyond what they show. Coverage is therefore read per definition and per gap, never from the number of tests. Composition is the one definition with no command, and the bounded goal is carried only approximately, by sessions (B.2).

One result here concerns the counter-case of Section 6.2 rather than a definition. A history-free local agent, using the participant package and the authorized sources, scored 10/10 on a frozen 10-point rubric without hard failure; Section 6.2 says what this single case does and does not show.

B.2 Sessions as approximate bounded goals

The prototype has no separate goal object: a session stands in for the bounded goal of Section 4.4. The command `ros today` opens the session and freezes its specification, which covers the purpose and its version, the target with its scope and digest, the rules and fields to preserve, the authority, the budget, and the sources with their cutoff. Running it again on the same day for the same target keeps that snapshot and reports what differs, and an admitted revision affects only the next goal. With `--for`, the session narrows to one commitment, its judgments, the cards that hold them, its source card and its basis cards. A target that does not exist or is ambiguous opens no session, and an empty or absent scope snapshot cannot report complete or complete_with_held (gap 16).

The budget covers boundaries, questions and reopenings and is frozen with the specification. Each boundary is charged to its session, including one at which only waiting continues, so the budget counts boundaries rather than elapsed time. Exhaustion stops questions, dispatch and further boundaries, but not the maintenance of requirements or the recording of reopenings, and a session then closed unfinished is budget_stopped unless old held questions exempt it. Gaps 19–22 record where metering and accounting are incomplete.

Waiting is exempt from completion only when the researcher put it there with a trigger that has not expired: by a triggered deferral or narrowing through the researcher's own channel, or by confirming `ros decide … defer` at the terminal. A deferral confirmed at the terminal therefore does produce waiting, while waiting on an out-of-scope dependency has no producer (gap 17). A deferral asserted off the terminal (`invoker_asserted`), an unconfirmed legacy deferral, an executor's deferral, a dependency inside the scope, and waiting without a trigger or past it grant no exemption.

Some things always block completion: an unreconstructable earlier specification, a break-glass block or field confirmation, in-scope unevaluated work, and a dispatched run without a usable result, including one that failed with unknown effects. A failure before dispatch does not. Answer-triggered holds can nonetheless rest on invalid old questions (gap 18). An explicit abandonment records the unknown effects without a receipt and lets the session settle.

At closure the session counts requirements, not inputs, and only in-scope unevaluated work affects completion. Early closure is recorded as `human_stopped`, all questions are listed regardless of scope, and an unverifiable receipt cannot exclude an existing in-scope requirement. Each boundary freezes enough of its inputs for a later boundary to tell a requirement missed under the earlier specification from one caused by a change. A cold start, or a missing earlier specification, is recorded as initial; in the second case the miss status is unknown, and one requirement for the unreconstructable specification is registered, or reopened if it was closed. Attribution of new requirements to scope, source and version changes is partial, and semantic caching is absent (gaps 23–25).

B.3 Code and tests

Sections 2 and 5 and this appendix describe the downloadable version at commit f541cb6d161a and the author's prototype at commit 4a9f696a6ebc. In CI, the downloadable version's 661 Python regression tests pass on CPython 3.10 and 3.13, and the default regression set of the author's prototype passes on CPython 3.11; tests of components not under development are outside these default sets. These are software checks of the stated commits and establish neither human acceptance nor scientific validity.

References

[1] Hollan, James; Hutchins, Edwin; Kirsh, David. Distributed Cognition: Toward a New Foundation for Human-Computer Interaction Research. ACM Transactions on Computer-Human Interaction, 2000. https://doi.org/10.1145/353485.353487.

[2] Cheney, James; Ahmed, Amal; Acar, Umut A. Provenance as dependency analysis. Mathematical Structures in Computer Science 21(6):1301–1337, 2011. https://doi.org/10.1017/S0960129511000211.

[3] MacLean, Allan; Young, Richard M.; Bellotti, Victoria M. E. et al. Questions, Options, and Criteria: Elements of Design Space Analysis. Human–Computer Interaction 6(3–4), 201–250, 1991. https://doi.org/10.1207/s15327051hci0603&4_2.

[4] Singh, Jatinder; Cobbe, Jennifer; Norval, Chris. Decision Provenance: Harnessing Data Flow for Accountable Systems. IEEE Access 7:6562–6574, 2019. https://doi.org/10.1109/ACCESS.2018.2887201.

[5] Buçinca, Zana; Malaya, Maja Barbara; Gajos, Krzysztof Z. To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-assisted Decision-making. Proceedings of the ACM on Human-Computer Interaction, 2021. https://doi.org/10.1145/3449287.

[6] Ma, Yiping; Li, Xinjin; Hu, Shiyu et al. Trustworthy AI in education: Framework, cases, and governance strategies. Innovation and Emerging Technologies, 2025. https://doi.org/10.1142/S2737599425500264.

[7] Clark, Herbert H.; Brennan, Susan E. Grounding in communication. Perspectives on Socially Shared Cognition, 1991. https://doi.org/10.1037/10096-006.

[8] Simon, Herbert A. A Behavioral Model of Rational Choice. The Quarterly Journal of Economics 69(1):99–118, 1955. https://doi.org/10.2307/1884852.

[9] Malone, Thomas W.; Crowston, Kevin. The Interdisciplinary Study of Coordination. ACM Computing Surveys 26(1), 87–119, 1994. https://ccs.mit.edu/papers/CCSWP157.html.

[10] Lappas, Theodoros; Liu, Kun; Terzi, Evimaria. Finding a team of experts in social networks. Proceedings of KDD 2009, pp. 467–476. https://cs-people.bu.edu/evimaria/cs591/lappas09finding.pdf.

[11] Argyris, Chris. Single-Loop and Double-Loop Models in Research on Decision Making. Administrative Science Quarterly, 1976. https://doi.org/10.2307/2391848.

[12] Kleppmann, Martin; Wiggins, Adam; van Hardenberg, Peter et al. Local-first software: you own your data, in spite of the cloud. Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software. https://doi.org/10.1145/3359591.3359737.

[13] Wilson, Greg; Bryan, Jennifer; Cranston, Karen et al. Good enough practices in scientific computing. PLOS Computational Biology, 2017. https://doi.org/10.1371/journal.pcbi.1005510.

[14] Parnas, D. L. On the criteria to be used in decomposing systems into modules. Communications of the ACM, 1972. https://doi.org/10.1145/361598.361623.

[15] Walsh, James P.; Ungson, Gerardo Rivera. Organizational Memory. Academy of Management Review, 1991. https://doi.org/10.5465/amr.1991.4278992.

[16] March, James G. Exploration and Exploitation in Organizational Learning. Organization Science 2(1):71–87, 1991. https://doi.org/10.1287/orsc.2.1.71.

[17] Yan, Lixiang; Greiff, Samuel; Teuber, Ziwen et al. Promises and challenges of generative artificial intelligence for human learning. Nature Human Behaviour, 2024. https://doi.org/10.1038/s41562-024-02004-5.

[18] Flavell, John H. Metacognition and cognitive monitoring: A new area of cognitive-developmental inquiry. American Psychologist, 1979. https://doi.org/10.1037/0003-066X.34.10.906.

[19] Glickman, Moshe; Sharot, Tali. How human–AI feedback loops alter human perceptual, emotional and social judgements. Nature Human Behaviour, 2024. https://doi.org/10.1038/s41562-024-02077-2.

[20] Vaccaro, Michelle; Almaatouq, Abdullah; Malone, Thomas. When combinations of humans and AI are useful: A systematic review and meta-analysis. Nature Human Behaviour, 2024. https://doi.org/10.1038/s41562-024-02024-1.

[21] Mitchell, Margaret; Wu, Simone; Zaldivar, Andrew et al. Model Cards for Model Reporting. ACM FAT* 2019. https://doi.org/10.1145/3287560.3287596.

[22] Ribeiro, Marco Tulio; Wu, Tongshuang; Guestrin, Carlos et al. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. ACL 2020:4902–4912. https://doi.org/10.18653/v1/2020.acl-main.442.

[23] Hu, Shiyu; Zhang, Dailing; Wu, Meiqi et al. A Multi-modal Global Instance Tracking Benchmark (MGIT): Better Locating Target in Complex Spatio-temporal and Causal Relationship. Advances in Neural Information Processing Systems 36, 2023. https://proceedings.nips.cc/paper_files/paper/2023/hash/4ea14e6090343523ddcd5d3ca449695f-Abstract-Datasets_and_Benchmarks.html.

[24] Li, Xuchen; Li, Xuzhao; Hu, Shiyu et al. CausalStep: A Benchmark for Explicit Stepwise Causal Reasoning in Videos. AAAI 40(8):6530–6538, 2026. https://doi.org/10.1609/aaai.v40i8.37582.

[25] Li, Xuchen; Li, Xuzhao; Hu, Shiyu et al. Select Less, Reason More: Prioritizing Evidence Purity for Video Reasoning. CVPR 2026. https://openaccess.thecvf.com/content/CVPR2026/html/Li_Select_Less_Reason_More_Prioritizing_Evidence_Purity_for_Video_Reasoning_CVPR_2026_paper.html.

[26] Hu, Shiyu; Li, Xuchen; Li, Xuzhao et al. FIOVA: A Multi-Annotator Benchmark for Human-Aligned Video Captioning. arXiv preprint, 2025. arXiv:2410.15270. https://arxiv.org/abs/2410.15270.

[27] Abadi, Martin; Lamport, Leslie. The Existence of Refinement Mappings. Theoretical Computer Science 82(2), pp. 253–284, 1991. https://research.google/pubs/the-existence-of-refinement-mappings-2/.

[28] Horvitz, Eric. Principles of Mixed-Initiative User Interfaces. Proceedings of CHI 1999. https://doi.org/10.1145/302979.303030.

[29] Madaan, Aman; Tandon, Niket; Gupta, Prakhar et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023. https://doi.org/10.52202/075280-2019.

[30] Shinn, Noah; Cassano, Federico; Gopinath, Ashwin et al. Reflexion: language agents with verbal reinforcement learning. NeurIPS 2023. https://doi.org/10.52202/075280-0377.

[31] Heckel, Reiko. Graph Transformation in a Nutshell. Electronic Notes in Theoretical Computer Science 148(1):187–198, 2006. https://doi.org/10.1016/j.entcs.2005.12.018.

[32] Kung, H. T.; Robinson, John T. On Optimistic Methods for Concurrency Control. ACM Transactions on Database Systems 6(2), 1981. https://doi.org/10.1145/319566.319567.

[33] Parker Jr., D. Stott; Popek, Gerald J.; Rudisin, Gerard et al. Detection of Mutual Inconsistency in Distributed Systems. IEEE Transactions on Software Engineering SE-9(3), 1983. https://doi.org/10.1109/TSE.1983.236733.

[34] Doyle, Jon. A Truth Maintenance System. Artificial Intelligence 12(3), 1979. https://doi.org/10.1016/0004-3702(79)90008-0.

[35] McAllester, David. Truth Maintenance. Proceedings of the Eighth National Conference on Artificial Intelligence (AAAI-90), 1109–1116, 1990. https://cdn.aaai.org/AAAI/1990/AAAI90-164.pdf.

[36] de Kleer, Johan. An assumption-based TMS. Artificial Intelligence 28(2), 127–162, 1986. https://doi.org/10.1016/0004-3702(86)90080-9.

[37] Alchourrón, Carlos E.; Gärdenfors, Peter; Makinson, David. On the logic of theory change: Partial meet contraction and revision functions. Journal of Symbolic Logic 50(2), 510–530, 1985. https://doi.org/10.2307/2274239.

[38] Mokhov, Andrey; Mitchell, Neil; Peyton Jones, Simon. Build Systems à la Carte. Proceedings of the ACM on Programming Languages 2(ICFP), 2018. https://doi.org/10.1145/3236774.

[39] Rasmussen, Preston; Paliychuk, Pavlo; Beauvais, Travis et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv preprint, 2025. arXiv:2501.13956. https://arxiv.org/abs/2501.13956.

[40] Gazit, Tiferet. Building an agentic memory system for GitHub Copilot. Engineering report, The GitHub Blog, 2026. https://github.blog/ai-and-ml/github-copilot/building-an-agentic-memory-system-for-github-copilot/.

[41] Wang, Yiqi; Liu, Jiaqi; Zhang, Jiaqi et al. PACMI: Provenance-Aware Cascading Memory Invalidation for Long-Term LLM Agents. arXiv preprint, 2026. arXiv:2610.05732. https://arxiv.org/abs/2610.05732.

[42] Qi, Yijiashun; Xu, Xiang; Li, Yuxuan. When Not to Write Memory: Governing False Promotion from Correlated Agent Traces. arXiv preprint, 2026. arXiv:2607.02579. https://arxiv.org/abs/2607.02579.

[43] Schmidgall, Samuel; Su, Yusheng; Wang, Ze et al. Agent Laboratory: Using LLM Agents as Research Assistants. Findings of EMNLP 2025:5977–6043. https://doi.org/10.18653/v1/2025.findings-emnlp.320.

[44] Mitchener, Ludovico; Yiu, Angela; Chang, Benjamin et al. Kosmos: An AI Scientist for Autonomous Discovery. arXiv preprint, 2025. https://arxiv.org/abs/2511.02824.

[45] Jaradeh, Mohamad Yaser; Oelen, Allard; Farfar, Kheir Eddine et al. Open Research Knowledge Graph: Next Generation Infrastructure for Semantic Scholarly Knowledge. Proceedings of the 10th International Conference on Knowledge Capture (K-CAP 2019). https://doi.org/10.1145/3360901.3364435.

[46] IBM. Link validity. Software documentation, IBM Engineering Lifecycle Management 7.0.3, 2026. https://www.ibm.com/docs/en/engineering-lifecycle-management-suite/lifecycle-management/7.0.3?topic=management-link-validity.

[47] GitHub. About protected branches. Software documentation, GitHub Docs, 2026. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches.

[48] Brooker, Joanne; Synnot, Anneliese; McDonald, Steve et al. Guidance for the production and publication of Cochrane living systematic reviews: Cochrane Reviews in living mode. Cochrane guidance, Version 1.0, December 2019. https://resources.cochrane.org/sites/resources.cochrane.org/files/uploads/inline-files/Transform/201912_LSR_Revised_Guidance.pdf.

[49] Cochrane. Cochrane strengthens systems to manage retracted publications in its published reviews. News release, Cochrane, 2026. https://www.cochrane.org/about-us/news/cochrane-strengthens-systems-manage-retracted-publications-its-published-reviews.

[50] Magee, Liam. Geist in the Machine: Simulating Recognition and Inner Dialogue in AI-Mediated Teaching and Research. arXiv preprint, 2026. arXiv:2603.10450. https://arxiv.org/abs/2603.10450.

[51] Zhang, Bailing. Provenance-Native Audit Infrastructure for LLM-Maintained Wiki Knowledge Systems. Artificial Intelligence and Emerging Technologies 3(3), 10, 2026. https://doi.org/10.53941/aiet.2026.100010.

[52] Shao, Chenyang; Huang, Dehao; Li, Yu et al. OmniScientist: Toward a Co-evolving Ecosystem of Human and AI Scientists. arXiv preprint, 2025. https://arxiv.org/abs/2511.16931.

[53] Jiang, Tao; Navet, Benjamin; Taghzouti, Yousouf et al. Magnolia: A Persistent-Memory Agent Framework for Computational Chemistry. arXiv preprint, 2026. arXiv:2610.27230. https://arxiv.org/abs/2610.27230.

[54] Xia, Yihan; Wang, Taotao. ResearchLoop: An Evidence-Gated Control Plane for AI-Assisted Research. arXiv preprint, 2026. arXiv:2605.28282. https://arxiv.org/abs/2605.28282.

[55] Engelbart, Douglas C. Augmenting Human Intellect: A Conceptual Framework. SRI Summary Report AFOSR-3223, 1962. https://www.dougengelbart.org/content/view/138/.

[56] Amershi, Saleema; Weld, Dan; Vorvoreanu, Mihaela et al. Guidelines for Human-AI Interaction. Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. https://doi.org/10.1145/3290605.3300233.

[57] Wu, Tongshuang; Terry, Michael; Cai, Carrie Jun. AI Chains: Transparent and Controllable Human-AI Interaction by Chaining Large Language Model Prompts. ACM CHI 2022. https://doi.org/10.1145/3491102.3517582.

[58] Acar, Umut A.; Blume, Matthias; Donham, Jacob. A Consistent Semantics of Self-adjusting Computation. ESOP 2007, 458–474. https://doi.org/10.1007/978-3-540-71316-6_31.

[59] Jiang, Yucheng; Shao, Yijia; Ma, Dekun et al. Into the Unknown Unknowns: Engaged Human Learning through Participation in Language Model Agent Conversations. EMNLP 2024. https://doi.org/10.18653/v1/2024.emnlp-main.554.

[60] Litt, Geoffrey; Schoening, Max; Shen, Paul et al. Potluck: Dynamic documents as personal software. Ink & Switch research essay, 2022. https://www.inkandswitch.com/potluck/.

[61] Chan, Joel; Akamatsu, Matthew; Vargas, David et al. Steps Towards an Infrastructure for Scholarly Synthesis. arXiv preprint, 2024. arXiv:2407.20666. https://arxiv.org/abs/2407.20666.

[62] Xu, Wujiang; Liang, Zujie; Mei, Kai et al. A-Mem: Agentic Memory for LLM Agents. NeurIPS 2025. https://doi.org/10.52202/085713-0593.

[63] Young, Justin. Effective harnesses for long-running agents. Engineering report, 2025. https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents.

[64] Rajasekaran, Prithvi. Harness design for long-running application development. Anthropic Engineering report, 2026. https://www.anthropic.com/engineering/harness-design-long-running-apps.

[65] Cemri, Mert; Pan, Melissa Z; Yang, Shuyi et al. Why Do Multi-Agent LLM Systems Fail? NeurIPS 2025 Datasets and Benchmarks Track. https://doi.org/10.52202/085713-4082.

[66] Anthropic. Claude Code release v2.1.277. Software release notes, 2026. https://github.com/anthropics/claude-code/releases/tag/v2.1.277.

[67] Wei, Jason; Wang, Xuezhi; Schuurmans, Dale et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. https://doi.org/10.52202/068431-1800.

[68] Wang, Xuezhi; Wei, Jason; Schuurmans, Dale et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023. https://openreview.net/forum?id=1PL1NIMMrw.

[69] Yao, Shunyu; Yu, Dian; Zhao, Jeffrey et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 2023. https://doi.org/10.52202/075280-0517.

[70] Yao, Shunyu; Zhao, Jeffrey; Yu, Dian et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. https://openreview.net/forum?id=WE_vluYUL-X.

[71] Schick, Timo; Dwivedi-Yu, Jane; Dessi, Roberto et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023. https://doi.org/10.52202/075280-2997.

[72] Patil, Shishir G.; Zhang, Tianjun; Wang, Xin et al. Gorilla: Large Language Model Connected with Massive APIs. NeurIPS 2024. https://doi.org/10.52202/079017-4020.

[73] Wu, Qingyun; Bansal, Gagan; Zhang, Jieyu et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. COLM 2024. https://openreview.net/forum?id=BAakY1hNKS.

[74] Hong, Sirui; Zhuge, Mingchen; Chen, Jonathan et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. ICLR 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/6507b115562bb0a305f1958ccc87355a-Abstract-Conference.html.

[75] Qian, Chen; Liu, Wei; Liu, Hongzhang et al. ChatDev: Communicative Agents for Software Development. ACL 2024. https://doi.org/10.18653/v1/2024.acl-long.810.

[76] Gao, Dawei; Li, Zitao; Pan, Xuchen et al. AgentScope: A Flexible yet Robust Multi-Agent Platform. arXiv preprint, 2024. arXiv:2402.14034. https://arxiv.org/abs/2402.14034.

[77] Wang, Guanzhi; Xie, Yuqi; Jiang, Yunfan et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. Transactions on Machine Learning Research, 2024. https://openreview.net/forum?id=ehfRiF0R3a.

[78] Bommasani, Rishi; Hudson, Drew A.; Adeli, Ehsan et al. On the Opportunities and Risks of Foundation Models. arXiv preprint, 2022. arXiv:2108.07258. https://arxiv.org/abs/2108.07258.

[79] Karpukhin, Vladimir; Oguz, Barlas; Min, Sewon et al. Dense Passage Retrieval for Open-Domain Question Answering. EMNLP 2020. https://doi.org/10.18653/v1/2020.emnlp-main.550.

[80] Guu, Kelvin; Lee, Kenton; Tung, Zora et al. Retrieval Augmented Language Model Pre-Training. ICML 2020; PMLR 119:3929–3938. https://proceedings.mlr.press/v119/guu20a.html.

[81] Lewis, Patrick; Perez, Ethan; Piktus, Aleksandra et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. https://papers.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

[82] Izacard, Gautier; Grave, Edouard. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. EACL 2021. https://doi.org/10.18653/v1/2021.eacl-main.74.

[83] Borgeaud, Sebastian; Mensch, Arthur; Hoffmann, Jordan et al. Improving Language Models by Retrieving from Trillions of Tokens. ICML 2022; PMLR 162:2206–2240. https://proceedings.mlr.press/v162/borgeaud22a.html.

[84] Izacard, Gautier; Lewis, Patrick; Lomeli, Maria et al. Atlas: Few-shot Learning with Retrieval Augmented Language Models. Journal of Machine Learning Research 24(251):1–43, 2023. https://jmlr.org/papers/v24/23-0037.html.

[85] Gao, Yunfan; Xiong, Yun; Gao, Xinyu et al. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint, 2024. arXiv:2312.10997. https://arxiv.org/abs/2312.10997.

[86] Trivedi, Harsh; Balasubramanian, Niranjan; Khot, Tushar et al. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. ACL 2023. https://doi.org/10.18653/v1/2023.acl-long.557.

[87] Asai, Akari; Wu, Zeqiu; Wang, Yizhong et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/25f7be9694d7b32d5cc670927b8091e1-Abstract-Conference.html.

[88] Zhang, Tianjun; Patil, Shishir G.; Jain, Naman et al. RAFT: Adapting Language Model to Domain Specific RAG. COLM 2024. https://openreview.net/forum?id=rzQGHXNReU.

[89] Chen, Tong; Wang, Hongwei; Chen, Sihao et al. Dense X Retrieval: What Retrieval Granularity Should We Use? EMNLP 2024. https://doi.org/10.18653/v1/2024.emnlp-main.845.

[90] Edge, Darren; Trinh, Ha; Cheng, Newman et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv preprint, 2025. arXiv:2404.16130. https://arxiv.org/abs/2404.16130.

[91] Gutiérrez, Bernal J.; Shu, Yiheng; Gu, Yu et al. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. NeurIPS 2024. https://doi.org/10.52202/079017-1902.

[92] Packer, Charles; Wooders, Sarah; Lin, Kevin et al. MemGPT: Towards LLMs as Operating Systems. arXiv preprint, 2024. arXiv:2310.08560. https://arxiv.org/abs/2310.08560.

[93] Zhong, Wanjun; Guo, Lianghong; Gao, Qiqi et al. MemoryBank: Enhancing Large Language Models with Long-Term Memory. AAAI 38(17):19724–19731, 2024. https://doi.org/10.1609/aaai.v38i17.29946.

[94] Park, Joon Sung; O'Brien, Joseph; Cai, Carrie Jun et al. Generative Agents: Interactive Simulacra of Human Behavior. ACM UIST 2023. https://doi.org/10.1145/3586183.3606763.

[95] Chhikara, Prateek; Khant, Dev; Aryan, Saket et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. ECAI 2025:2993–3000. https://doi.org/10.3233/FAIA251160.

[96] Letta. MemFS. Software documentation, Letta Docs, 2026. https://docs.letta.com/concepts/memfs.

[97] Letta. Letta Code v0.33.0. Software release notes, GitHub, 2026. https://github.com/letta-ai/letta-code/releases/tag/v0.33.0.

[98] Sumers, Theodore R.; Yao, Shunyu; Narasimhan, Karthik et al. Cognitive Architectures for Language Agents. Transactions on Machine Learning Research, 2024. https://openreview.net/forum?id=1i6ZCyf1QJ.

[99] Hu, Yuyang; Liu, Shichun; Yue, Yanwei et al. Memory in the Age of AI Agents. arXiv preprint, 2026. arXiv:2512.13564. https://arxiv.org/abs/2512.13564.

[100] Du, Pengfei. Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers. arXiv preprint, 2026. arXiv:2603.07670. https://arxiv.org/abs/2603.07670.

[101] Wu, Ming; Zhu, Pengyuan. Agent Zero Memory: Provenance-Aware Long-Term Memory for LLM Agents. arXiv preprint, 2026. arXiv:2608.29606. https://arxiv.org/abs/2608.29606.

[102] Wilkinson, Mark D.; Dumontier, Michel; Aalbersberg, IJsbrand Jan et al. The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, 2016. https://doi.org/10.1038/sdata.2016.18.

[103] Soiland-Reyes, Stian; Sefton, Peter; Crosas, Mercè et al. Packaging research artefacts with RO-Crate. Data Science 5(2):97–138, 2022. https://doi.org/10.3233/DS-210053.

[104] RO-Crate Community. RO-Crate Metadata Specification 1.3.0. Specification, 2026. doi:10.5281/zenodo.20720080. https://www.researchobject.org/ro-crate/specification/1.3/.

[105] Greshake, Kai; Abdelnabi, Sahar; Mishra, Shailesh et al. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. ACM AISec 2023 Workshop. https://doi.org/10.1145/3605764.3623985.

[106] Zhan, Qiusi; Liang, Zhixiang; Ying, Zifan et al. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. Findings of ACL 2024. https://doi.org/10.18653/v1/2024.findings-acl.624.

[107] Debenedetti, Edoardo; Zhang, Jie; Balunovic, Mislav et al. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. NeurIPS 2024 Datasets and Benchmarks Track. https://doi.org/10.52202/079017-2636.

[108] Cerruti, Tommaso; Okamoto, Mika; Erol, Ansel Kaplan. Agent Memory Is a Surface for Endogenous Authorization Laundering. arXiv preprint, 2026. arXiv:2609.01836. https://arxiv.org/abs/2609.01836.

[109] Hu, Shiyu; Zhao, Xin; Huang, Lianghua et al. Global Instance Tracking: Locating Target More Like Humans. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2023. https://doi.org/10.1109/TPAMI.2022.3153312.

[110] Zhao, Xin; Hu, Shiyu; Wang, Yipei et al. BioDrone: A Bionic Drone-Based Single Object Tracking Benchmark for Robust Vision. International Journal of Computer Vision, 2024. https://doi.org/10.1007/s11263-023-01937-0.

[111] Hu, Shiyu; Zhao, Xin; Huang, Kaiqi. SOTVerse: A User-Defined Task Space of Single Object Tracking. International Journal of Computer Vision, 2024. https://doi.org/10.1007/s11263-023-01908-5.

[112] Kramer, Stefan; Cerrato, Mattia; Brugger, Jannis et al. Automated Scientific Discovery: From Equation Discovery to Autonomous Discovery Systems. Machine Learning 115:109, 2026. https://doi.org/10.1007/s10994-025-06955-2.

[113] Bran, Andres M.; Cox, Sam; Schilter, Oliver et al. Augmenting large language models with chemistry tools. Nature Machine Intelligence 6:525–535, 2024. https://doi.org/10.1038/s42256-024-00832-8.

[114] Boiko, Daniil A.; MacKnight, Robert; Kline, Ben et al. Autonomous chemical research with large language models. Nature, 2023. https://doi.org/10.1038/s41586-023-06792-0.

[115] Lu, Chris; Lu, Cong; Lange, Robert Tjarko et al. Towards end-to-end automation of AI research. Nature 651:914–919, 2026. https://doi.org/10.1038/s41586-026-10265-5.

[116] Gottweis, Juraj; Weng, Wei-Hung; Daryin, Alexander et al. Accelerating scientific discovery with Co-Scientist. Nature 655:487–496, 2026. https://doi.org/10.1038/s41586-026-10644-y.

[117] Ghafarollahi, Alireza; Buehler, Markus J. SciAgents: Automating Scientific Discovery Through Bioinspired Multi-Agent Intelligent Graph Reasoning. Advanced Materials 37(22):2413523, 2025. https://doi.org/10.1002/adma.202413523.

[118] Ghareeb, Ali E.; Chang, Benjamin; Mitchener, Ludovico et al. A multi-agent system for automating scientific discovery. Nature 655:497–505, 2026. https://doi.org/10.1038/s41586-026-10652-y.

[119] Luo, Jixiang. XScientist: A Git-Like Research Protocol for Long-Running Autonomous Scientific Discovery. arXiv preprint, 2026. arXiv:2607.12301. https://arxiv.org/abs/2607.12301.

[120] Ma, Yiping; Hu, Shiyu; Li, Xuchen et al. When LLMs Learn to be Students: The SOEI Framework for Modeling and Evaluating Virtual Student Agents in Educational Interaction. arXiv preprint, 2025. arXiv:2410.15701. https://arxiv.org/abs/2410.15701.

[121] Ma, Yiping; Hu, Shiyu; Zhu, Buyuan et al. EduVerse: A User-Defined Multi-Agent Simulation Space for Education Scenario. arXiv preprint, 2025. arXiv:2510.05650. https://arxiv.org/abs/2510.05650.

[122] Zhu, Buyuan; Hu, Shiyu; Ma, Yiping et al. EduPersona: Benchmarking Subjective Ability Boundaries of Virtual Student Agents. arXiv preprint, 2025. arXiv:2510.04648. https://arxiv.org/abs/2510.04648.

[123] Liu, Xiao; Yu, Hao; Zhang, Hanchen et al. AgentBench: Evaluating LLMs as Agents. ICLR 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/e9df36b21ff4ee211a8b71ee8b7e9f57-Abstract-Conference.html.

[124] Ma, Chang; Zhang, Junlei; Zhu, Zhihao et al. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. NeurIPS 2024 Datasets and Benchmarks Track. https://doi.org/10.52202/079017-2365.

[125] Min, Sewon; Krishna, Kalpesh; Lyu, Xinxi et al. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. EMNLP 2023:12076–12100. https://doi.org/10.18653/v1/2023.emnlp-main.741.

[126] Gao, Tianyu; Yen, Howard; Yu, Jiatong et al. Enabling Large Language Models to Generate Text with Citations. EMNLP 2023:6465–6488. https://doi.org/10.18653/v1/2023.emnlp-main.398.

[127] Es, Shahul; James, Jithin; Espinosa Anke, Luis et al. RAGAs: Automated Evaluation of Retrieval Augmented Generation. EACL 2024 System Demonstrations. https://doi.org/10.18653/v1/2024.eacl-demo.16.

[128] Chen, Jiawei; Lin, Hongyu; Han, Xianpei et al. Benchmarking Large Language Models in Retrieval-Augmented Generation. AAAI 38(16):17754–17762, 2024. https://doi.org/10.1609/aaai.v38i16.29728.

[129] Yang, Xiao; Sun, Kai; Xin, Hao et al. CRAG - Comprehensive RAG Benchmark. NeurIPS 2024 Datasets and Benchmarks Track. https://doi.org/10.52202/079017-0335.

[130] Zheng, Lianmin; Chiang, Wei-Lin; Sheng, Ying et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023 Datasets and Benchmarks Track. https://doi.org/10.52202/075280-2020.

[131] Li, Xuzhao; Li, Xuchen; Hu, Shiyu et al. VerifyBench: A Systematic Benchmark for Evaluating Reasoning Verifiers Across Domains. AAAI 40(38), 2026. https://doi.org/10.1609/aaai.v40i38.40448.

[132] Burton, Jason W.; Lopez-Lopez, Ezequiel; Hechtlinger, Shahar et al. How large language models can reshape collective intelligence. Nature Human Behaviour, 2024. https://doi.org/10.1038/s41562-024-01959-9.

[133] Li, Zhiyu; Song, Shichao; Wang, Hanyu et al. MemOS: An Operating System for Memory-Augmented Generation (MAG) in Large Language Models. arXiv preprint, 2025. https://arxiv.org/abs/2505.22101.

[134] Asai, Akari; He, Jacqueline; Shao, Rulin et al. Synthesizing scientific literature with retrieval-augmented language models. Nature, 2026. https://doi.org/10.1038/s41586-025-10072-4.

[135] Skarlinski, Michael D.; Cox, Sam; Laurent, Jon M. et al. Language agents achieve superhuman synthesis of scientific knowledge. arXiv preprint, 2024. https://arxiv.org/abs/2409.13740.

[136] Villaescusa-Navarro, Francisco; Bolliet, Boris; Villanueva-Domingo, Pablo et al. The Denario project: Deep knowledge AI agents for scientific discovery. arXiv preprint, 2025. https://arxiv.org/abs/2510.26887.

[137] Lyu, Yougang; Zhang, Xi; Yi, Xinhao et al. EvoScientist: Towards Multi-Agent Evolving AI Scientists for End-to-End Scientific Discovery. arXiv preprint, 2026. https://arxiv.org/abs/2603.08127.

[138] Marshall, Catherine C.; Shipman III, Frank M.; Coombs, James H. VIKI: Spatial Hypertext Supporting Emergent Structure. Proceedings of ECHT '94, pp. 13–23, 1994. https://people.engr.tamu.edu/shipman/abstracts/echt94-abstract.html.